7. ÜNİTE — MANTIKSAL AJANLAR

Eksiksiz konu sırası • Türkçe çeviri ve açıklamalı ders notları

Yapay Zekâya Giriş — Öğrenci Ders Notu

Ekim 2026

İçindekiler

Bu Ders Notu Nasıl Kullanılmalı? **3**
İçerik Haritası . . . 3

7 — Mantıksal Ajanlar (Logical Agents) **4**
7.1 — Bilgi Tabanlı Ajanlar (Knowledge-Based Agents) . . . 4
7.2 — Wumpus Dünyası (The Wumpus World) . . . 6

7.3 — Mantık (Logic) **10**
Sözdizimi ve anlambilim . . . 10
Olası dünya, model, doyurma . . . 10
Mantıksal gerektirme (Entailment) . . . 10
Mantıksal çıkarım, model denetimi ve ispat gösterimi . . . 12

7.4 — Önermeler Mantığı: Çok Basit Bir Mantık **14**
7.4.1 — Sözdizimi . . . 14
7.4.2 — Anlambilim (Semantics) . . . 15
7.4.3 — Basit Bir Wumpus Bilgi Tabanı . . . 17
7.4.4 — Basit Bir Çıkarım Yöntemi . . . 17

7.5 — Önermeler Mantığında Teorem İspatlama **21**
Geçerlilik, doyurulabilirlik, çelişkiyle ispat . . . 21
7.5.1 — Çıkarım ve ispatlar . . . 22
7.5.2 — Resolution ile İspat . . . 23
7.5.3 — Horn Clause'lar ve Kesin Clause'lar . . . 26
7.5.4 — İleri ve Geri Zincirleme . . . 27

7.6 — Etkili Önermeler Mantığı Model Denetimi **30**
7.6.1 — Tam bir geri izleme algoritması: DPLL . . . 30
7.6.2 — Yerel arama ve WalkSAT . . . 32
7.6.3 — Rastgele SAT Problemlerinin Görünümü . . . 33

7.7 — Önermeler Mantığına Dayalı Ajanlar **35**
7.7.1 — Dünyanın mevcut durumu (The current state) . . . 35
7.7.2 — Hibrit ajan (A hybrid agent) . . . 38
7.7.3 — Mantıksal Durum Tahmini (Logical State Estimation) . . . 39
7.7.4 — Önermesel Çıkarımla Plan Yapma . . . 41
7. Ünitenin Tek Paragraflık Sonucu . . . 42

EK A — Temel Kavramlar ve Sembol Sözlüğü **43**
Formül Rehberi . . . 44

EK B — Adım Adım Çözümlü Alıştırmalar **45**
Hızlı Tekrar İçin Sınav Öncesi Kontrol Listesi . . . 48

Bu Ders Notu Nasıl Kullanılmalı?

Bu doküman, Stuart Russell ve Peter Norvig'in *Artificial Intelligence: A Modern Approach* (3. baskı) kitabının **7. ünitesinin, kullanıcı tarafından paylaşılan İngilizce metinlere dayanılarak hazırlanmış Türkçe çeviri ve açıklamalı ders notu** sürümüdür. Kitaptaki **7.1–7.7** konu sırası korunmuştur. Gönderilen özgün şekiller ilgili başlıklarda kullanılmış, metindeki ancak ayrıca resim olarak gönderilmeyen algoritma şekilleri Türkçe sözde kod olarak yeniden dizilmiştir. Kaynak açıklamaları ile yeni öğrenmeyi kolaylaştırıcı örnekler, "Öğretici örnek" veya "Ders notu" gibi ifadelerle ayırt edilmiştir.

Önemli: Mantıksal gösterimde $\models$ "her uygun modelde doğru olma" ilişkisini, $\vdash$ ise belirli bir ispat yöntemiyle türetmeyi anlatır. $\neg$: değil; $\wedge$: ve; $\vee$: veya; $\Rightarrow$: ise; $\Leftrightarrow$: ancak ve ancak. Wumpus haritasında koordinatlar **[sütun, satır]** sırasındadır; [1,1] sol alt köşedir.

İçerik Haritası

- **7.1 Bilgi tabanlı ajanlar:** bilgi tabanı, TELL/ASK, çıkarım, bilgi düzeyi, bildirimsel yaklaşım.
- **7.2 Wumpus dünyası:** PEAS, algılar, güvenlik, beş adımlı çıkarım, Şekil 7.2–7.4.
- **7.3 Mantık:** sözdizimi, anlambilim, model, gerektirme, sağlamlık, tamlık, temellendirme.
- **7.4 Önermeler mantığı:** semboller, bağlaçlar, doğruluk tabloları, basit KB, model denetimi.
- **7.5 Teorem ispatlama:** tutarlılık, doyurulabilirlik, Modus Ponens, Resolution, CNF, Horn cümleleri, zincirleme.
- **7.6 Etkili model denetimi:** DPLL, WalkSAT, rastgele SAT problemlerinin eşik davranışı.
- **7.7 Mantığa dayalı ajanlar:** zamanla değişen durum, frame problem, hibrit ajan, belief state, SATPLAN.
- **Ekler:** sembol sözlüğü, matematiksel dönüşüm rehberi, alıştırmalar ve açıklamalı yanıtlar.

7 — Mantıksal Ajanlar (Logical Agents)

Bu bölümde, karmaşık dünyaların temsillerini oluşturabilen; çıkarım (inference) yoluyla dünya hakkında yeni temsiller türetebilen ve bu temsillerden yararlanarak ne yapacaklarına karar verebilen ajanlar tasarlıyoruz. İnsanlar bir şeyler bilir ve bildikleri şeyler eylemlerine yön verir. Bu gözlem önemsiz değildir: insan zekâsının yalnızca reflekslerden değil, bilginin içsel temsilleri üzerinde işleyen **akıl yürütmeden (reasoning)** doğduğunu ileri sürer. Yapay zekâdaki karşılığı **bilgi tabanlı ajan**dır.

3. ve 4. bölümlerdeki problem çözen ajanların da bir tür bilgisi vardır; ancak bu bilgi sınırlı ve esnekliği düşük bir biçimde tutulur. Örneğin 8-bulmaca için bir hareketin sonucu, probleme özel RESULT fonksiyonunun koduna gömülüdür. Ajan bu fonksiyonla hareketin sonucunu hesaplayabilir; fakat iki taşın aynı konumu dolduramayacağını veya belirli parite sınıfları arasında ulaşılamazlık olduğunu kolayca ayrıca çıkaramaz. Durumu tek bir atom olarak temsil etmek, kısmen gözlemlenebilir büyük ortamlarda daha da yetersizdir: o anda mümkün olan tüm somut durumları listelemek gerekebilir.
4. bölümdeki **değişkenlere değer atama** yaklaşımı daha iyidir. Bu bölümde bu fikri mantıksal gösterimlere taşıyoruz. Mantık, bilgi tabanlı ajanların parçaları birleştirip yeniden düzenleyerek farklı amaçlarla kullanabilecekleri genel bir temsil ailesidir. Matematikçinin teorem ispatlaması veya astronomun çok uzak bir geleceği hesaplaması gibi, çıkarımlar anlık ihtiyaçlardan bağımsız bile olabilir. Bilgi tabanlı ajanlar yeni hedefleri açıkça kabul edebilir, kendilerine verilen veya öğrendikleri bilgilerle hızla yetkinleşebilir, çevre değişince ilgili bilgilerini güncelleyebilir.

Önce ajanın genel mimarisini, sonra Wumpus dünyası üzerinden mantıksal kararları inceleyeceğiz. Ardından mantığın genel ilkelerini, önermeler mantığını, çıkarım teknolojilerini ve bunların Wumpus ajanında birleşmesini öğreneceğiz. Önermeler mantığı, sonraki ünitedeki birinci dereceden mantık kadar ifade gücüne sahip olmasa da mantığın temel kavramlarını göstermeye yeterlidir.

7.1 — Bilgi Tabanlı Ajanlar (Knowledge-Based Agents)

Bilgi tabanı, cümleler ve aksiyomlar

Bilgi tabanlı ajanın merkezinde **bilgi tabanı** (*knowledge base*, KB) bulunur. KB, **cümleler (sentences)** kümesidir. Buradaki *cümle* teknik anlam taşır: doğal dildeki dilbilgisel cümleyle ilişkili olsa da onunla aynı şey değildir. Her cümle, bir **bilgi temsil dilinde (knowledge representation language)** ifade edilir ve dünya hakkında bir iddia temsil eder. Başka cümlelerden türetilmeden başlangıçta kabul edilen bir cümleye **aksiyom (axiom)** denilebilir.

İki temel işlem gerekir: **TELL**, bilgi tabanına yeni cümle ekler; **ASK**, bilgi tabanında bilinenleri sorgular. İki işlem de daha eski cümlelerden yenilerini türetme anlamındaki **çıkarımı** içerebilir. Bir sorgunun cevabı, daha önce bildirilen bilgilerden gerçekten çıkmalıdır; çıkarım mekanizması cevapları uydurmamalıdır.

Öğretici örnek: KB'de "Her insan ölümlüdür" ve "Sokrates insandır" bilgileri varsa "Sokrates ölümlüdür" sonucunu ayrıca başlangıçta yazmadan elde edebiliriz. Bunun geçerliliği, öncüllerin doğru kabul edilmesine bağlıdır.

Bilgi tabanlı ajanın çalışma döngüsü — Şekil 7.1

Şekil 7.1'deki genel ajan programı bir **algı** alır, bir **eylem** döndürür ve kalıcı bir KB ile zaman sayacı tutar. Her çağrıda: (1) algının belirli zamanda yaşandığı KB'ye bildirilir, (2) yapılacak eylem sorgulanır, (3) seçilen eylemin gerçekleştirildiği KB'ye kaydedilir, (4) zaman artırılır. Sorgu sırasında mevcut dünya, olası eylem dizileri ve sonuçları hakkında kapsamlı akıl yürütme yapılabilir.



Şekil 7.1 — Genel bilgi tabanlı ajan döngüsü (yeniden çizilmiş öğretici görsel).

```
function KB-AGENT(percept) -> action
  kalıcı: KB = bilgi tabanı
          t  = 0  // zaman sayacı
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action <- ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t <- t + 1
  return action
```

`MAKE-PERCEPT-SENTENCE`, belirli algıyı ve zamanı mantıksal cümleye çevirir. `MAKE-ACTION-QUERY`, "şu anda hangi eylem?" sorusunu oluşturur. `MAKE-ACTION-SENTENCE`, gerçekleşen eylemi belirtir. Temsil dilinin ayrıntısı bu arayüzlerin; çıkarım tekniğinin ayrıntısı ise TELL/ASK'ın içinde saklıdır.

Bilgi düzeyi ile uygulama düzeyi

Bu ajan 2. bölümdeki iç duruma sahip ajanlara benzer. Fark, TELL ve ASK tanımları sayesinde davranışını **bilgi düzeyinde (knowledge level)** tanımlayabilmemizdir: neyi bildiğini ve neyi hedeflediğini söyleyerek yapacağı hareketi açıklayabiliriz. Örneğin otonom bir taksinin San Francisco'dan Marin County'ye gitme hedefi ve Golden Gate Köprüsü'nün iki yer arasındaki tek bağlantı olduğu bilgisi varsa köprüyü kullanmasını bekleriz. Bunun bağlı listelerle, piksel haritalarıyla, yazmaçlarda sembol işlemekle veya sinir ağlarıyla nasıl gerçekleştirildiği **uygulama düzeyi (implementation level)** sorusudur.

Bildirimsel ve prosedürel yaklaşım; öğrenme

Boş bilgi tabanına ajan için gerekli cümleleri tek tek TELL ederek davranışı yapılandırmak **bildirimsel yaklaşım (declarative approach)** olarak adlandırılır. **Prosedürel yaklaşım (procedural approach)** ise davranışları doğrudan program koduna işler. 1970'ler ve 1980'lerde bu iki yaklaşım tartışılmıştır; bugün iyi ajanların genellikle ikisini birleştirdiği ve bildirimsel bilginin bazen verimli prosedürel koda dönüştürülebildiği anlaşılmıştır. Ajana öğrenme mekanizmaları eklenirse algılardan çevreye ilişkin genel bilgi üretilebilir; böyle bir öğrenen ajan tam özerkliğe erişebilir.

Ders örneği — robot süpürge: “Önümde duvar var” bilgisi TELL ile saklanır. “Hangi eylemi seçmeliyim?” sorusu ASK ile yönlendirilir. “Duvar varsa ileri gitme” kuralı ve uygun dönüş olasılıkları değerlendirilir. Son olarak seçilen dönüş ve zaman KB’ye kaydedilir. Burada algıyı kaydetmek ile algıdan yeni bilgi üretmek farklı işlemlerdir.

Kontrol sorusu: TELL ve ASK arasındaki fark nedir? **Kısa cevap:** TELL bilgi ekler; ASK KB’nin desteklediği cevabı çıkarmaya çalışır.

7.2 — Wumpus Dünyası (The Wumpus World)

Mağara ve tehlikeler

Bilgi tabanlı ajanların yararını göstermek için Wumpus dünyası adında bir mağara tanımlanır. Geçitlerle bağlanan odaların birinde, içeri gireni yiyen Wumpus vardır. Ajanın Wumpus’u vurmak için bir oku bulunur. Bazı odalar dipsiz çukur içerir; içine giren herkes düşer, yalnızca kocaman Wumpus düşmeyecek kadar büyüktür. Mağaranın ödülü ise bir yığın altındır. Modern oyunlara göre basit olsa da bu dünya zekânın önemli yönlerini gösterir.

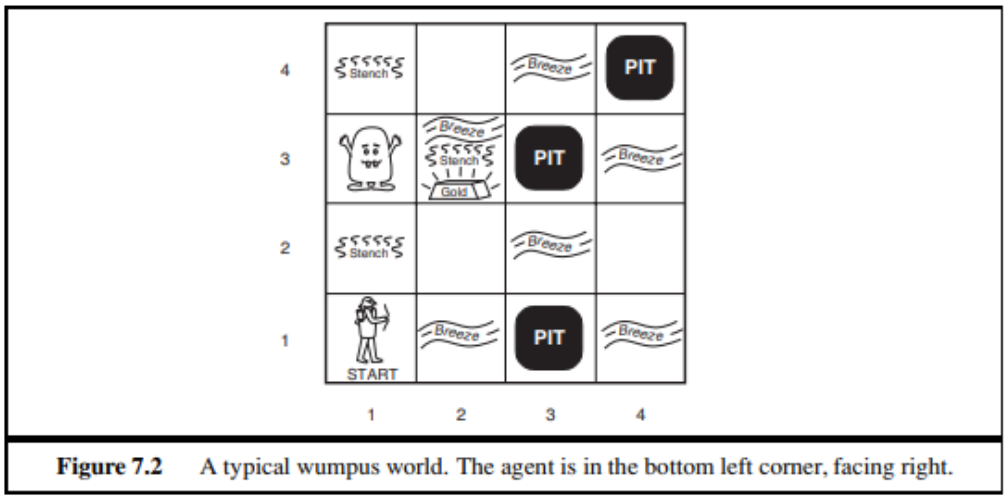


Figure 7.2 A typical wumpus world. The agent is in the bottom left corner, facing right.

Şekil 7.2 — Tipik Wumpus dünyası. Ajan sol alt [1,1] karesinde ve sağa dönüktür; görsel, gerçek dünya düzenini gösterir. Ajanın başlangıçta tüm haritayı bildiği varsayılmaz.

Harita 4 × 4 = 16 kare içerir. Gönderilen örnek dünyada Wumpus **[1,3]**, altın **[2,3]**, çukurlar **[3,1], [3,3], [4,4]** konumlarındadır. Koku ve esinti sembolleri duyusal ipuçlarıdır; çapraz komşuluk sayılmaz.

PEAS: performans, ortam, eyleyici, algılayıcı

P — Performans (Performance measure). Ajan altını alıp mağaradan çıkarsa +1000, çukura düşer veya Wumpus’a yenilirse −1000, her eylem için −1 ve oku kullanırsa ek −10 puan alır. Oyun ölüm veya mağaradan çıkışla sona erer. Dolayısıyla hem güvenlik hem kısa eylem dizisi önemlidir.

E — Ortam (Environment). 4 × 4 karelik ızgara; başlangıç karesi [1,1] ve başlangıç yönü doğudur. Altın ve Wumpus başlangıç dışındaki karelerden ayrı ayrı düzgün dağılımla rastgele seçilir. Başlangıç dışındaki her kare 0,2 olasılıkla çukur olabilir. Bu son ifade “tam olarak her beş kareden biri çukurdur” demek değildir.

A — Eyleyiciler (Actuators). `Forward` bir kare ilerler; `TurnLeft` ve `TurnRight` 90 derece döndürür. Duvarın önünde `Forward` hareket yaratmaz. Canlı Wumpus’a veya çukura giren ajan ölür; ölü Wumpus’un olduğu odaya girebilir. `Grab` aynı karedeki altını alır. `Shoot` baktığı yönde düz

çizgide ok atar; ok Wumpus'a ya da duvara çarpana kadar gider. Yalnızca bir ok vardır; sonraki atışlar etkisizdir. `Climb` yalnızca [1,1]'den çıkış sağlar.

S — Algılayıcılar (Sensors). Beş adet tek-bitlik algı kullanılır: **Stench** (Wumpus'un kendi karesinde veya doğrudan komşusunda koku), **Breeze** (çukura doğrudan komşu karede esinti), **Glitter** (altının bulunduğu karede parıltı), **Bump** (duvara çarpma), **Scream** (Wumpus ölünce mağaranın her yerinden duyulabilen çığlık). Algı vektörü şu sıradadır:

`[Stench, Breeze, Glitter, Bump, Scream]`

Örneğin `[Stench, Breeze, None, None, None]` "koku ve esinti var, diğerleri yok" demektir.

Ortamın özellikleri

Bu dünya **ayrık**, **statik** (Wumpus hareket etmez), **tek ajanlı**, **sıralı** (ödül birçok eylemden sonra gelebilir) ve **kısmen gözlemlenebilir** bir ortamdır. Ajanın kendi konumu, Wumpus'un sağlığı ve okun bulunup bulunmadığı gibi durumların doğrudan algılanması yerine, bunlar eylem geçmişiyle izlenir. Çukur ve Wumpus yerleri bilinmeyen ama sabit durum unsurları sayılırsa geçiş modelinin genel kuralları bilinir; hangi `Forward` hareketlerinin ölümcül olduğunu henüz bilmemek ise modelin eksik bilinmesi şeklinde de yorumlanabilir.

Başlangıçtaki temel sorun ortam düzeninin bilinmemesidir; bunu aşmak mantıksal akıl yürütmeyi gerektirir. Çoğu haritada altın güvenle alınabilir; bazı durumlarda ajan eli boş dönmek ile ölümü göze almak arasında karar vermelidir. Kaynaktaki rastgele ortam tanımında yaklaşık %21'i "adaletsiz"dir: altın çukurda veya çukurlarla çevrilidir.

İlk algı ve ilk karar — Şekil 7.3

Ajanın başlangıç KB'si dünya kurallarını ve [1,1]'in güvenli olduğunu içerir. Haritalarda **A** ajanı, **OK** güvenli kareyi, **V** ziyaret edilmiş kareyi, **P?** olası çukuru, **P!** kesin çukuru, **W!** kesin Wumpus'u belirtir.

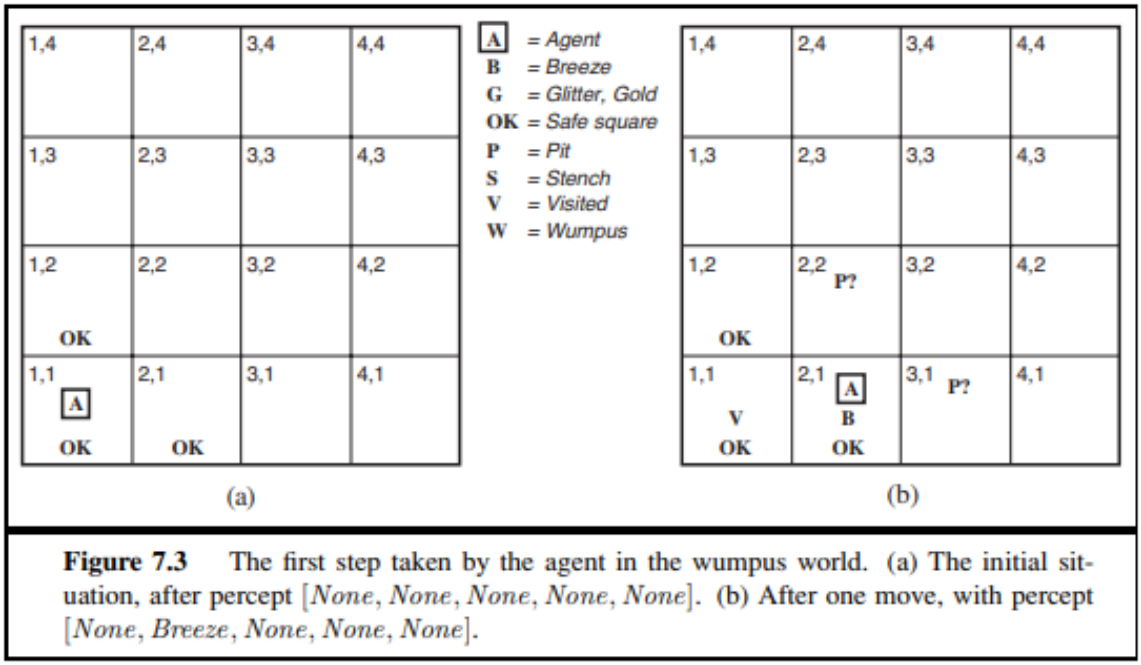


Figure 7.3 The first step taken by the agent in the wumpus world. (a) The initial situation, after percept [*None*, *None*, *None*, *None*, *None*]. (b) After one move, with percept [*None*, *Breeze*, *None*, *None*, *None*].

Şekil 7.3 — (a) [1,1]'de ilk algı `[None,None,None,None,None]`; (b) [2,1]'de bir hareket sonrası algı `[None,Breeze,None,None,None]`.

1. [1,1]'de ne koku ne esinti vardır. O hâlde doğrudan komşular [1,2] ve [2,1]'de **Wumpus da çukur da yoktur**; ikisi OK olarak işaretlenir. Tedbirli ajan yalnızca güvenli olduğu bilinen

karelere girer.

2. Ajan [2,1]'e ilerler; burada **esinti** vardır. Komşuları [1,1], [2,2] ve [3,1]'dir. Başlangıç karesi çukur içeremediğinden [2,2] veya [3,1] ya da her ikisi çukur içermelidir. P? kesin bilgi değil olasılık işaretidir.

Bu son akıl yürütme şu şekilde yazılır:

$$B_{2,1} \wedge \neg P_{1,1} \implies P_{2,2} \vee P_{3,1}.$$

Geri dönme, Wumpus'u ve çukuru bulma — Şekil 7.4

Ajanın ziyaret edilmemiş ve güvenliği kanıtlanmış tek komşusu [1,2]'dir. Bu nedenle dönüp [1,1]'e, oradan [1,2]'ye gider. Burada **koku var ama esinti yok**: `[Stench,None,None,None,None]`.

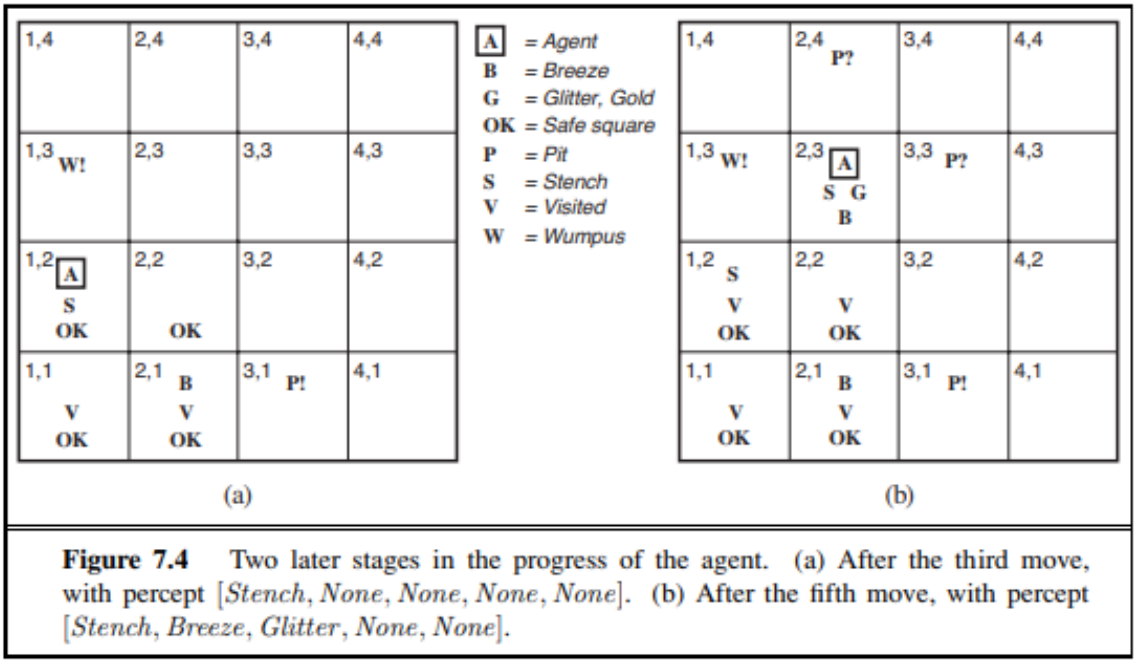


Figure 7.4 Two later stages in the progress of the agent. (a) After the third move, with percept [*Stench*, *None*, *None*, *None*, *None*]. (b) After the fifth move, with percept [*Stench*, *Breeze*, *Glitter*, *None*, *None*].

Şekil 7.4 — (a) Üçüncü yer değiştirmeden sonra koku var/esinti yok. (b) Beşinci yer değiştirmeden sonra `[Stench,Breeze,Glitter,None,None]`; ajan altının karesindedir.

Wumpus çıkarımı: [1,2]'nin komşuları [1,1], [1,3], [2,2]'dir. Başlangıç [1,1] güvenlidir. [2,2]'de Wumpus olsaydı, ajan [2,1]'de de koku alırdı; orada koku almadı. Geriye **[1,3]** kalır: `W!`.

Çukur çıkarımı: [1,2]'de esinti olmaması, [2,2]'de çukur olmadığını gösterir. Oysa [2,1]'deki esintiden $P_{2,2} \vee P_{3,1}$ çıkarılmıştı. Demek ki **[3,1] kesin çukurdur**: `P!`. Burada farklı yer ve zamanlardan gelen bilgiler, özellikle **bir algının yokluğu**, belirleyici rol oynar.

Güvenli geçiş ve altın: [2,2]'de Wumpus da çukur da yoktur; ajan buradan [2,3]'e ilerler. Parıltı algıladığı için altını alır ve güvenli yolu izleyerek [1,1]'e dönüp çıkmalıdır. Bu bölümün asıl dersi şudur: Veriler doğruysa ve çıkarım kuralları sound ise türetilen sonuç, bilginin doğru olduğu her dünyada doğru olur.

Aşama	Konum	Algı	Kesin ya da kısmi çıkarım
1	[1,1]	Koku ve esinti yok	[1,2] ve [2,1] güvenli
2	[2,1]	Esinti var	[2,2] veya [3,1] çukur
3	[1,2]	Koku var; esinti yok	Wumpus [1,3]; çukur [3,1]
4	[2,2]	Önceki çıkarımlarla güvenli	[2,3]'e ilerlenebilir
5	[2,3]	Koku, esinti, parıltı	Altın burada: `Grab` ve dönüş

Ders sorusu: [2,1]’de esinti olmasa [3,1] için hangi yeni sonuç çıkarılabilirdi? **Yanıt:** Esinti bulunmaması, komşu [3,1]’de çukur bulunmadığını da kanıtlardı.

7.3 — Mantık (Logic)

Bu bölüm mantıksal gösterim ve akıl yürütmenin, herhangi bir belirli mantık biçiminden bağımsız kavramlarını özetler. Teknik ayrıntılara geçmeden önce örnek olarak sıradan aritmetiği kullanacağız.

Sözdizimi ve anlambilim

Bilgi tabanı, temsil dilinin **sözdizimi (syntax)** tarafından izin verilen, iyi biçimlendirilmiş cümleler içerir. x+y=4 aritmetik açısından iyi biçimli, x4y+= ise kötü biçimli bir dizidir. Bir mantık dili aynı zamanda **anlambilim (semantics)** tanımlamalıdır: cümle hangi dünyada doğru, hangisinde yanlıştır? x+y=4, x=2, y=2 iken doğru; x=1, y=1 iken yanlıştır. Klasik mantıkta her cümle belirli bir dünyada doğru ya da yanlıştır; ara değer yoktur. (Bulanık mantık 14. bölümde farklı doğruluk derecelerini ele alır.)

Kritik ayrım: 5+3=10 düzgün yazılmış fakat **yanlış** bir önermedir. 5+=3 10 ise doğru biçimli bir aritmetik cümle değildir. Sözdizimsel geçerlilik, anlamsal doğruluk değildir.

Olası dünya, model, doyurma

Olası dünya (possible world), ajanın içinde bulunabileceği veya bulunmayabileceği bir çevre olarak düşünülebilir. Daha kesin olmak için **model (model)** deriz: ilgili cümlelerin doğruluklarını belirleyen matematiksel değer ataması. Örneğin briç oynayan masada x erkek ve y kadın varsa x+y=4 ifadesi toplam dört kişi bulunan olası dünyalarda doğrudur. Matematiksel model, x ile y için belirli gerçek sayılar seçilmesidir.

Bir m modelinde α doğruysa, **m, α'yı sağlar (satisfies)** denir. Bir cümlenin doğru olduğu bütün modeller:

$$M(\alpha) = \{m \mid m \text{ modelinde } \alpha \text{ doğrudur}\}$$

ile gösterilir. x+y=4 için (1,3), (2,2) ve (0,4) uygun örneklerdir; (1,1) değildir. Söz konusu sayıları gerçek sayılar olarak aldığımızda uygun model sayısı sonsuzdur.

Mantıksal gerektirme (Entailment)

Bir α cümlesinin β sonucunu **mantıksal olarak gerektirmesi**, α'nın doğru olduğu her modelde β'nın da doğru olmasıdır:

$$\boxed{\alpha \models \beta \iff M(\alpha) \subseteq M(\beta).}$$

Alt küme işaretinin yönü önemlidir: α, β'dan daha güçlü bilgi verebilir ve daha çok olası dünyayı eleyebilir. Örneğin x=0 bilgisi, xy=0 sonucunu gerektirir; y ne olursa olsun çarpım sıfırdır. Fakat xy=0, x=0 sonucunu gerektirmez: x=5,y=0 karşı örnektir.

Şekil 7.5: Sekiz model ve kesin/kararsız çıkarım

Önceki Wumpus örneğinde [1,1]'de **hiçbir algı yok**, [2,1]'de **esinti var**. Bu iki algı ve dünya kuralları birlikte KB'yi oluşturur. Yalnızca [1,2], [2,2], [3,1] karelerinde çukur olup olmadığını incelersek üç ikili değişken için $2^3 = 8$ model vardır.

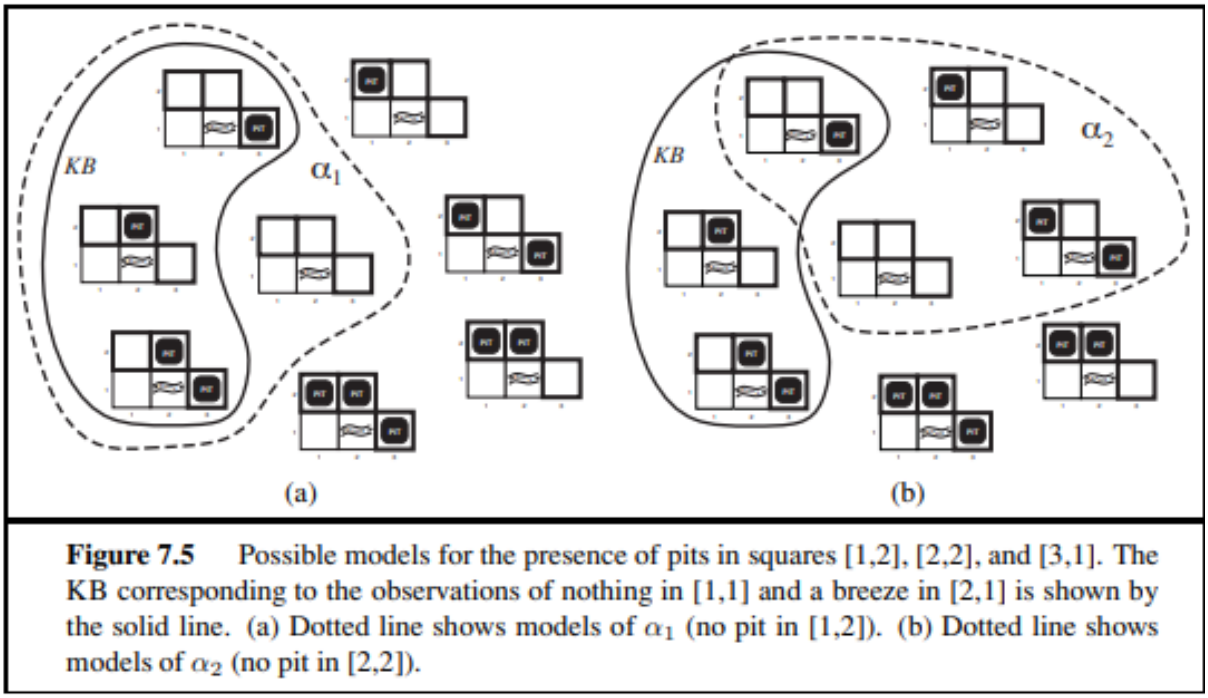


Figure 7.5 Possible models for the presence of pits in squares [1,2], [2,2], and [3,1]. The KB corresponding to the observations of nothing in [1,1] and a breeze in [2,1] is shown by the solid line. (a) Dotted line shows models of α_1 (no pit in [1,2]). (b) Dotted line shows models of α_2 (no pit in [2,2]).

Şekil 7.5 — Kesintisiz sınır, algılarla uyumlu üç KB modelini; kesikli sınırlar sırasıyla $\alpha_1 = \neg P_{1,2}$ ve $\alpha_2 = \neg P_{2,2}$ koşullarını sağlayan modelleri çevreler.

Üç çukur değişkeni → 2³ = 8 model; KB ile uyumlu olanlar yeşil

M1	M2	M3	M4	M5	M6	M7	M8
$P_{1,2}$=0	$P_{1,2}$=0	$P_{1,2}$=0	$P_{1,2}$=0	$P_{1,2}$=1	$P_{1,2}$=1	$P_{1,2}$=1	$P_{1,2}$=1
$P_{2,2}$=0	$P_{2,2}$=0	$P_{2,2}$=1	$P_{2,2}$=1	$P_{2,2}$=0	$P_{2,2}$=0	$P_{2,2}$=1	$P_{2,2}$=1
$P_{3,1}$=0	$P_{3,1}$=1	$P_{3,1}$=0	$P_{3,1}$=1	$P_{3,1}$=0	$P_{3,1}$=1	$P_{3,1}$=0	$P_{3,1}$=1
ELENDİ	UYGUN	UYGUN	UYGUN	ELENDİ	ELENDİ	ELENDİ	ELENDİ

Öğretici ek şekil — 0: çukur yok; 1: çukur var. Yeşil olan M2, M3, M4 bilgi tabanıyla uyumludur.

Bu sekiz modelin ancak üçü KB'yi sağlar. [1,1]'de esinti olmadığı için $P_{1,2}$ yanlış olmalı; [2,1]'de esinti olduğundan $P_{2,2} \vee P_{3,1}$ doğru olmalıdır.

KB ile uyumlu model	$P_{1,2}$	$P_{2,2}$	$P_{3,1}$
M2	Yanlış	Yanlış	Doğru
M3	Yanlış	Doğru	Yanlış
M4	Yanlış	Doğru	Doğru

$\alpha_1 = \neg P_{1,2}$ bütün KB modellerinde doğrudur; o hâlde $KB \models \alpha_1$. $\alpha_2 = \neg P_{2,2}$ ise M3/M4'te yanlıştır; $KB \not\models \alpha_2$. Aynı nedenle $P_{2,2}$ de kesin çıkarılamaz. **Bilinmiyor** ile **yanlış** aynı şey değildir.

Bu örnekteki modeller, matematiksel olarak önerme sembollerine doğru/yanlış atamalarından başka bir şey değildir; görseldeki mağara kareleri yalnızca kolay anlaşılır bir çizimdir.

Mantıksal çıkarım, model denetimi ve ispat gösterimi

Mantıksal gerektirme, sonucun bilgilerden zorunlu olarak çıkması ilişkisidir; **mantıksal çıkarım (logical inference)** ise bu sonucu elde etme işlemidir. Şekil 7.5, bütün KB modellerini dolaşıp hedefi test eden **model denetimini (model checking)** gösterir. Başka bir deyişle $M(KB) \subseteq M(\alpha)$ doğrulanır.

Kitaptaki benzetmede KB'nin bütün sonuçları bir saman yığını, aranan sonuç α ise iğnedir: iğnenin yığının içinde **olması** gerektirme; onu **bulmak** çıkarımdır. Belirli bir `i` algoritmasının α'yı KB'den türettiğini:

$$KB \vdash_i \alpha$$

ile gösteririz. Bu işaret ⊨ ile aynı değildir; biri yöntemle türetmeyi, diğeri bütün modellerdeki zorunlu ilişkiyi anlatır.

Soundness ve completeness

Sağlamlık / doğruluğu koruma (soundness, truth-preserving): Algoritma yalnızca gerçekten gerektirilen sonuçları üretir:

$$KB \vdash_i \alpha \;\Rightarrow\; KB \models \alpha.$$

Tamlık (completeness): Gerektirilen her sonucu türetebilir:

$$KB \models \alpha \;\Rightarrow\; KB \vdash_i \alpha.$$

Sound olmayan çıkarım, olmayan bir iğne bulduğunu söylemek gibidir. Sonlu modeller üzerinde eksiksiz model denetimi sound'dur ve tamdır. Sonsuz model uzaylarında her modeli tek tek listelemek mümkün olmayabilir; bu nedenle tamlık sorunu önem kazanır. Sonsuz derinlikli aramada derinlik öncelikli aramanın her zaman tam olmamasıyla benzerlik kurulabilir. Bir çıkarım yöntemi sound ise ve KB gerçek dünyada doğruysa elde edilen sonuç da gerçek dünyada doğrudur.

Şekil 7.6: Temsil ile dünya arasında köprü

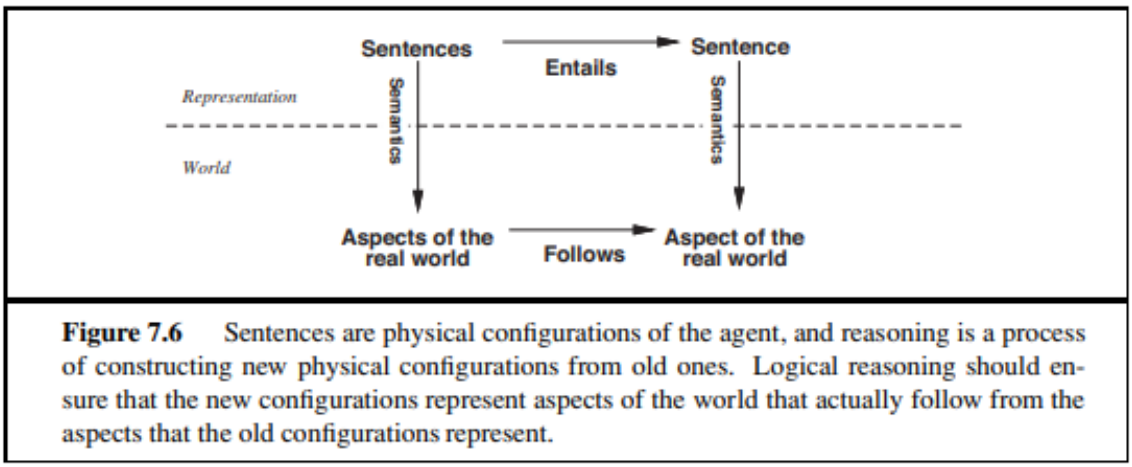


Figure 7.6 Sentences are physical configurations of the agent, and reasoning is a process of constructing new physical configurations from old ones. Logical reasoning should ensure that the new configurations represent aspects of the world that actually follow from the aspects that the old configurations represent.

Şekil 7.6 — Üst düzeyde cümlelerden çıkarımla yeni cümle üretilir; alt düzeyde bunların temsil ettiği gerçek dünya olguları arasında mantıksal ilişki vardır. Dikey "semantics" okları temsil ile gerçeklik arasındaki bağı belirtir.

Cümleler ajanın içinde fiziksel örüntüler olabilir: bilgisayarda bit dizileri veya beyinde elektriksel örüntüler. Çıkarım bunlar üzerinde sözdizimsel işlemler yapar; fakat sonuçların temsil ettiği dünya olgularının öncekilerden gerçekten çıkmasını istememizin nedeni anlambilimdir.

Temellendirme (Grounding)

KB bilgisinin gerçek çevreyle nasıl bağlantılandığı **temellendirme (grounding)** sorusudur: "KB'nin gerçek dünyada doğru olduğunu nereden biliyoruz?" Basit cevap sensörlerdir. Wumpus ajanı koku aldığında doğru çalışan sensör ve cümle oluşturma mekanizması, "burada koku var" ifadesini dünyaya bağlar. Ancak "Wumpus komşusunda koku üretir" gibi **genel kurallar**, tek bir algıya eşit değildir; öğrenmeyle elde edilmiş olabilir. Öğrenme hata yapabilir: örneğin Wumpus artık yılların 29 Şubat'ında banyo yaptığı için koku üretmiyorsa yalın koku kuralı her zaman doğru olmaz. KB'nin doğruluğu mantık tekniğinden değil, algıların, modellerin ve öğrenmenin güvenilirliğinden de etkilenir. Bu felsefi soru 26. bölümle; öğrenme ise kitabın beşinci kısmıyla bağlantılıdır.

7.4 — Önermeler Mantığı: Çok Basit Bir Mantık

Önermeler mantığı (propositional logic) basit ama güçlü bir biçimsel mantıktır. Önce hangi ifadelerin kurulabileceği (syntax), sonra hangi modellerde doğru oldukları (semantics), ardından gerektirme ve çıkarım incelenir. Wumpus dünyası uygulama örneğimiz olmaya devam eder.

7.4.1 — Sözdizimi

Atomik cümleler ve semboller

Atomik bir cümle tek bir **önerme sembolü**dür: `P`, `Q`, `R`, `W1,3`, `North` gibi. Her sembol doğru/yanlış değer alabilecek bir iddiayı temsil eder. $W_{1,3}$ "Wumpus [1,3]'tedir" anlamında seçilmiştir; `W`, `1`, `3` sembolün kendi başına ayrı ayrı anlamlı parçaları değildir. `True` her modelde doğru, `False` her modelde yanlış olan iki özel semboldür.

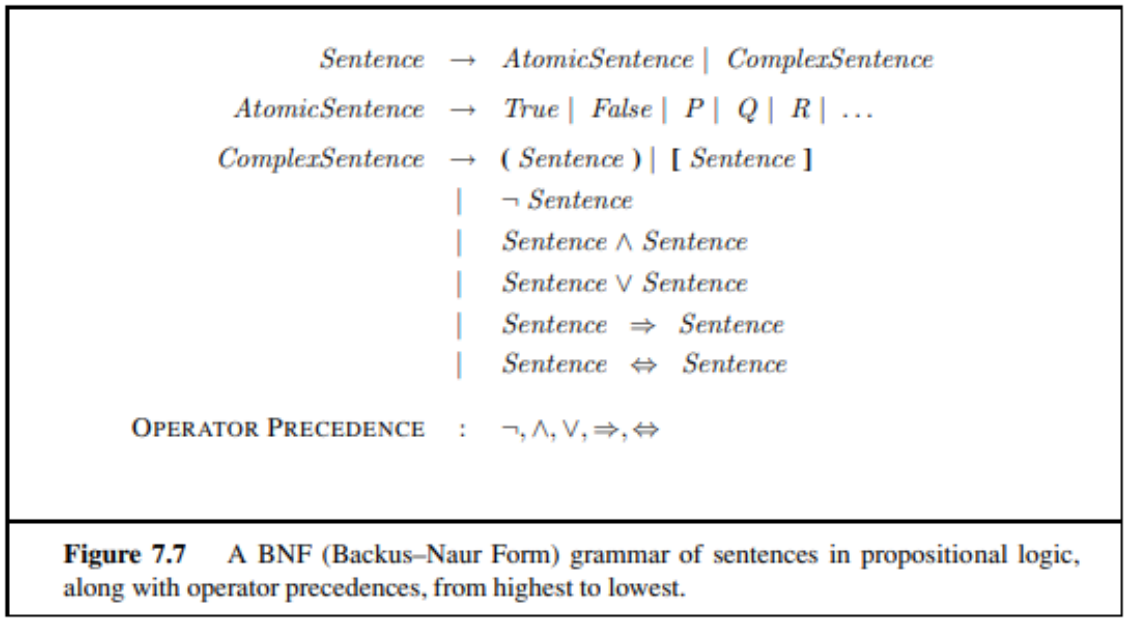
Literal, atomik önerme (pozitif literal) veya onun olumsuzlamasıdır (negatif literal). Örneğin P ve $\neg P$ birer literaldir. Bir önerme doğru/yanlış değeri alabilir; fakat ajanın bu değeri **bilmemesi** de mümkündür. Emir veya soru cümleleri tipik olarak önerme değildir.

Beş mantıksal bağlaç

Bağlaç	Adı	Mantıksal anlamı ve örnek
$\neg P$	Olumsuzlama (not)	P doğru değil. $\neg W_{1,3}$
$P \wedge Q$	Birleşim (and, conjunction)	İkisi de doğru. $W_{1,3} \wedge P_{3,1}$
$P \vee Q$	Ayrışım (or, disjunction)	En az biri doğru; ikisi de olabilir
$P \Rightarrow Q$	Koşullu önerme (implies)	"P ise Q"; P öncül, Q sonuç
$P \Leftrightarrow Q$	Çift koşullu (iff)	İki yönlü, "ancak ve ancak"

Örneğin `(W1,3 ∧ P3,1) ⇒ ¬W2,2` koşullu önermesinde sol taraf **öncül (premise/antecedent)**, sağ taraf **sonuç (conclusion/consequent)** olarak adlandırılır. Birleşimin bileşenlerine *conjunct*, ayrışımın bileşenlerine *disjunct* denir. Bazı kitaplar koşullu önerme için $\rightarrow$ veya $\supset$, çift koşullu için $\equiv$ işaretini kullanır.

Şekil 7.7: BNF dilbilgisi



$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \ldots \\
\textit{ComplexSentence} &\rightarrow (\ \textit{Sentence}\ ) \mid [\ \textit{Sentence}\ ] \\
&\mid \neg\ \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence}
\end{aligned}
$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

Şekil 7.7 — Önermeler mantığı için BNF (Backus–Naur Form) dilbilgisi ve operatör önceliği.

```
Sentence       -> AtomicSentence | ComplexSentence
AtomicSentence -> True | False | P | Q | R | ...
ComplexSentence-> ( Sentence ) | [ Sentence ]
                  | ¬ Sentence
                  | Sentence ∧ Sentence
                  | Sentence ∨ Sentence
                  | Sentence ⇒ Sentence
                  | Sentence ⇔ Sentence

Öncelik (yüksekten düşüğe): ¬, ∧, ∨, ⇒, ⇔
```

BNF'de | "alternatif" anlamındadır. İşlem önceliği tanımlanmazsa çok bağlaçlı cümleler birden fazla şekilde ayrıştırılabilir. $\neg A \wedge B$ ifadesi `(¬A ∧ B` demektir; `¬(A ∧ B)` değildir. Aritmetikte −2+4=2 olmasına benzer. Köşeli parantez ve normal parantez aynı işlevi görür; hangisinin seçildiği okunabilirlik içindir. **Öneri:** Öncelikten emin değilsen parantez kullan.

7.4.2 — Anlambilim (Semantics)

Model, her önerme sembolüne doğru veya yanlış değer atar. Örneğin:

$$m_1 = \{P_{1,2} = False,\ P_{2,2} = False,\ P_{3,1} = True\}.$$

Üç sembol $2^3 = 8$ model oluşturur. Bunlar fiziksel mağara olmaktan ziyade matematiksel değer atamalarıdır. $P_{1,2}$ sembolünün hangi dünya iddiasını anlattığı yorumla belirlenir; tek başına adı Wumpus ortamına zorunlu olarak bağlı değildir.

Atomik cümlelerin doğruluğu modelde doğrudan verilir; karmaşık cümleler için kurallar özyinelemeli uygulanır:

- $\neg P$ ancak `P` yanlışsa doğrudur.
- $P \wedge Q$ ancak ikisi de doğruysa doğrudur.

- $P \vee Q$ P veya Q ya da ikisi birden doğruysa doğrudur (**kapsayıcı VEYA**).
- $P \Rightarrow Q$ sadece **P doğru ve Q yanlışken** yanlıştır.
- $P \Leftrightarrow Q$ ikisinin doğruluk değeri aynıysa doğrudur.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Figure 7.8 Truth tables for the five logical connectives. To use the table to compute, for example, the value of $P \vee Q$ when P is true and Q is false, first look on the left for the row where P is *true* and Q is *false* (the third row). Then look in that row under the $P \vee Q$ column to see the result: *true*.

Şekil 7.8 — Beş temel mantıksal bağlacın doğruluk tablosu.

P	Q	¬P	P ∧ Q	P ∨ Q	P ⇒ Q	P ⇔ Q
Y	Y	D	Y	Y	D	D
Y	D	D	Y	D	D	Y
D	Y	Y	Y	D	Y	Y
D	D	Y	D	D	D	D

Tabloda D: doğru, Y: yanlış. Örneğin P doğru ve Q yanlışsa $P \vee Q$ doğrudur. $P \oplus Q$ (exclusive OR / XOR) ise ikisi birden doğru olduğunda **yanlış** olur; normal VEYA'dan farkı budur. XOR için kaynaklarda farklı semboller kullanılabilir.

Çözümlü örnek — model üzerinde cümle değerlendirme:

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$$

ifadesini yukarıdaki m_1 üzerinde değerlendirelim:

$$\neg False \wedge (False \vee True) = True \wedge True = \boxed{True}.$$

Bu işlemi cümle ağacı üzerinde her alt ifadeyi hesaplayacak biçimde yapan fonksiyona PL-TRUE?(s,m) adı verilir; 7.3 alıştırması bunu yazmayı ister.

Koşullu önerme neden sezgilerimizi şaşırtabilir?

Önermeler mantığında P ile Q arasında nedensel veya konusal bağ zorunlu değildir. "5 tekse Tokyo Japonya'nın başkentidir" (alışılmış yorumda) her iki taraf doğru olduğu için mantıksal olarak doğrudur. "5 çiftse Sam zekidir" ifadesinde öncül yanlış olduğu için Sam'in zekâsı ne olursa olsun koşullu ifade doğrudur. Çünkü $P \Rightarrow Q$, "P doğru olduğunda Q'nun doğruluğunu iddia ediyorum; P yanlışsa Q hakkında iddiada bulunmuyorum" anlamındadır.

Bir çift koşullu önerme $P \Leftrightarrow Q$, $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$ ile aynı anlama gelir. Wumpus esintisi kuralında çift koşul uygundur: "Birim kare esintilidir ancak ve ancak doğrudan komşusunda çukur varsa." Örneğin:

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

7.4.3 — Basit Bir Wumpus Bilgi Tabanı

Önce değişmeyen özellikleri temsil edelim: $P_{x,y}$ karede çukur, $W_{x,y}$ karede ölü veya diri Wumpus, $B_{x,y}$ esinti, $S_{x,y}$ koku var demektir. Şu beş cümle, Şekil 7.3(b)'deki ilk iki gözlemi temsil etmeye yeterlidir:

$$
\begin{aligned}
&R_1 : \neg P_{1,1} \\
&R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}) \\
&R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}) \\
&R_4 : \neg B_{1,1} \\
&R_5 : B_{2,1}.
\end{aligned}
$$

İlk üç cümle ortamın genel kurallarını ve başlangıç güvenliğini, son iki cümle bu dünyada yaşanmış algıları ifade eder. Bilgi tabanının toplamı, beşinin VE ile bağlanmasıdır: $KB = R_1 \wedge R_2 \wedge R_3 \wedge R_4 \wedge R_5$.

Elde çıkarım: R2 ile R4 birlikte, [1,2] ve [2,1]'de çukur olmadığını bildirir:

$$\neg(P_{1,2} \vee P_{2,1}) \equiv \neg P_{1,2} \wedge \neg P_{2,1}.$$

R3, R5 ve R1 birlikte, $P_{2,2} \vee P_{3,1}$ sonucunu verir: bu iki kareden en az biri çukurdur.

7.4.4 — Basit Bir Çıkarım Yöntemi

Soru $KB \models \alpha$ doğru mu? Doğrudan yöntem, KB ile sorguda geçen her sembolün doğru/yanlış atamalarını sıralamak ve KB'nin doğru olduğu her modelde α'nın da doğru olup olmadığını incelemektir.

Wumpus KB'sinde yedi sembol vardır: $B_{1,1}, B_{2,1}, P_{1,1}, P_{1,2}, P_{2,1}, P_{2,2}, P_{3,1}$. Bu nedenle $2^7 = 128$ model bulunur. Sadece **üçünde** R1–R5'in hepsi doğrudur; her üçünde de $P_{1,2} = False$ olduğu için $KB \models \neg P_{1,2}$. $P_{2,2}$ ise iki modelde doğru, bir modelde yanlıştır; henüz kesin bir sonuç yoktur.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Figure 7.9 A truth table constructed for the knowledge base given in the text. *KB* is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in [1,2]. On the other hand, there might (or might not) be a pit in [2,2].

Şekil 7.9 — 128 satırlı doğruluk tablosunun görünümü. Son sütundaki üç uygun satırda $P_{1,2}$ yanlıştır.

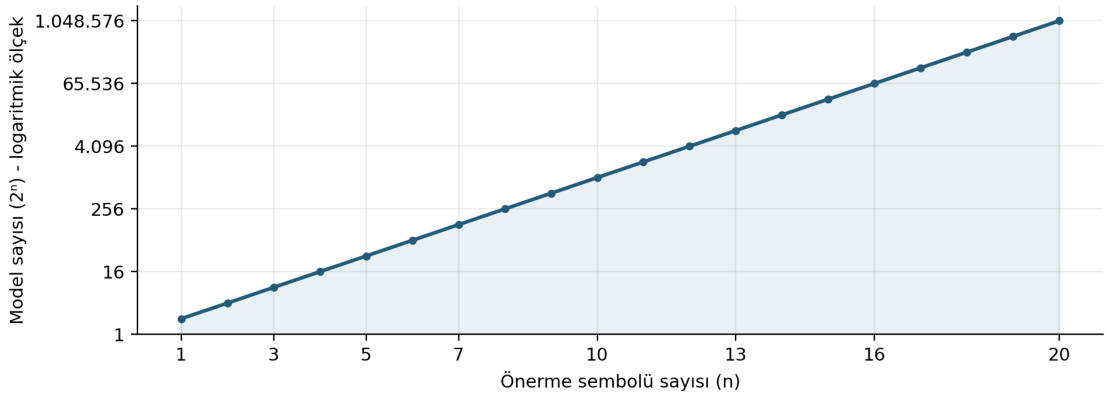
Şekil 7.10 — TT-ENTAILS? Algoritması

```
function TT-ENTAILS?(KB, alpha) -> true/false
  symbols <- KB ve alpha içindeki semboller
  return TT-CHECK-ALL(KB, alpha, symbols, {})

function TT-CHECK-ALL(KB, alpha, symbols, model)
  if symbols boşsa
    if PL-TRUE?(KB, model) then
      return PL-TRUE?(alpha, model)
    else
      return true  // KB'nin yanlış olduğu model ihlal yaratmaz
  else
    P <- FIRST(symbols)
    rest <- REST(symbols)
    return TT-CHECK-ALL(KB, alpha, rest, model U {P=true})
       and TT-CHECK-ALL(KB, alpha, rest, model U {P=false})
```

TT doğruluk tablosu demektir. Fonksiyonlar **kısmi modeller** üzerinde çalışır; tüm semboller atanınca tam model oluşur. KB'nin yanlış olduğu modelde `true` döndürülmesi bir hata değildir: gerektirme, sadece KB'nin doğru olduğu modelleri sınar. Algoritma hem **sound** (tanımı doğrudan uygular) hem **complete**tir (sonlu bütün atamaları bitirir).

Toplam n sembol için model sayısı 2^n olduğundan kitap algoritmanın **zaman karmaşıklığını** $O(2^n)$, derinlik öncelikli sayım sayesinde **bellek karmaşıklığını** $O(n)$ olarak verir; bunlar kullanılan gösterime ilişkin özet sınırdır.



Öğretici ek grafik — Sembol sayısı arttıkça incelenecek model sayısı üstel büyür. Dikey eksen 2 tabanında logaritmiktir.

Önermeler mantığında gerektirme genel durumda **co-NP-complete** bir problemdir; dolayısıyla bilinen genel amaçlı algoritmaların en kötü durum davranışı üstel olabilir. Sonraki bölümlerde birçok pratik örnekte daha verimli teknikler ele alınır.

Şekil 7.11 — Mantıksal eşdeğerlikler

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Figure 7.11 Standard logical equivalences. The symbols α, β, and γ stand for arbitrary sentences of propositional logic.

Şekil 7.11 — Değişme, birleşme, çift olumsuzlama, karşıt ters, koşul kaldırma, De Morgan ve dağılma eşdeğerlikleri.

Eşdeğerlik	Kural
$\alpha \wedge \beta \equiv \beta \wedge \alpha$	VE'nin değişmesi
$\alpha \vee \beta \equiv \beta \vee \alpha$	VEYA'nın değişmesi
$(\alpha \wedge \beta) \wedge \gamma \equiv \alpha \wedge (\beta \wedge \gamma)$	VE'nin birleşmesi
$(\alpha \vee \beta) \vee \gamma \equiv \alpha \vee (\beta \vee \gamma)$	VEYA'nın birleşmesi
$\neg\neg\alpha \equiv \alpha$	Çift olumsuzlama
$\alpha \Rightarrow \beta \equiv \neg\beta \Rightarrow \neg\alpha$	Karşıt tersleme
$\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$	İse'nin kaldırılması
$\alpha \Leftrightarrow \beta \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$	Çift koşulun kaldırılması
$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$	De Morgan
$\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$	De Morgan
$\alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$	VE'nin VEYA üzerine dağılımı
$\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$	VEYA'nın VE üzerine dağılımı

Eşdeğerlik	Kural

Öğretici örnek: $\neg(P \Rightarrow Q)$ ifadesi önce $\neg(\neg P \vee Q)$; ardından De Morgan ile $P \wedge \neg Q$ olur. Demek ki “P ise Q” yalnızca P doğru iken Q yanlışsa çürütülür.

7.5 — Önermeler Mantığında Teorem İspatlama

Model denetiminde, gerektirme ilişkisinin doğru olduğunu göstermek için modelleri sıralamıştık. **Teorem ispatlama (theorem proving)** ise bilgi tabanındaki cümlelere doğrudan **çıkarım kuralları** uygulayarak hedef cümlenin ispatını kurar; modelleri tek tek incelemek gerekmez. Model sayısı çok büyük, ispat zinciri kısa olduğunda teorem ispatlama daha verimli olabilir.

Geçerlilik, doyurulabilirlik, çelişkiyle ispat

İki cümle aynı model kümesinde doğruysa **mantıksal olarak eşdeğerdir**: $\alpha \equiv \beta$. Eşdeğerliğin diğer tanımı iki yönlü gerektirmedir: $\alpha \models \beta$ ve $\beta \models \alpha$. Şekil 7.11'deki mantıksal eşdeğerlikler, aritmetik özdeşlikler gibi dönüşümlerde işe yarar.

Geçerli (valid) cümle bütün modellerde doğrudur; **totoloji (tautology)** de denir. Örneğin $P \vee \neg P$ bütün atamalarda doğrudur, dolayısıyla True ile eşdeğerdir. Eski Yunan'dan beri bilinen tümdengelim teoremi:

$$\boxed{\alpha \models \beta \quad \iff \quad (\alpha \Rightarrow \beta) \text{ geçerlidir.}}$$

Bu nedenle gerektirmeyi, $\alpha \Rightarrow \beta$ cümlesinin geçerliliğini inceleyerek veya True ile eşdeğer olduğunu göstererek saptayabiliriz. Ters yönde, her geçerli koşullu önerme meşru bir çıkarım tanımlar (kitabın 7.5 alıştırması bu ilişkiyi ispatlatır).

Doyurulabilir (satisfiable) cümle, en az bir modelde doğrudur. Önceki R1∧ . . . ∧R5 bilgi tabanı üç modelde doğru olduğu için doyurulabilirdir. Bir cümleyi sağlayan model bulunamıyorsa **doyurulamaz (unsatisfiable)**dır. Önermeler mantığında doyurulabilirlik kararı **SAT** problemidir; NP-complete olduğu gösterilen ilk problem budur. Kısıt sağlama problemlerinin çoğu da uygun atama olup olmadığını sorar.

İlgili üç temel ilişki:

$$\begin{aligned} \alpha \text{ geçerli} &\iff \neg\alpha \text{ doyurulamaz,} \\ \alpha \text{ doyurulabilir} &\iff \neg\alpha \text{ geçerli değildir,} \\ \alpha \models \beta &\iff (\alpha \wedge \neg\beta) \text{ doyurulamaz.} \end{aligned}$$

Son ilişki **çelişkiyle ispat** (*reductio ad absurdum*, proof by refutation/contradiction) yöntemidir: β'nın yanlış olduğunu kabul eder; bunun bilinen α aksiyomlarıyla çeliştiğini gösteririz.

Çözümlü mini örnek: $P \Rightarrow Q$ ve P verilmişken Q'yu ispatlayalım. Tersini, $\neg Q$ kabul edelim: $(P \Rightarrow Q) \wedge P \wedge \neg Q \equiv (\neg P \vee Q) \wedge P \wedge \neg Q$. P doğru olduğundan ilk parantez Q gerektirir; fakat $\neg Q$ da vardır. $Q \wedge \neg Q$ çelişkisinden $KB \models Q$ çıkar.

7.5.1 — Çıkarım ve ispatlar

İspat, istenen hedefe varan sonuçlar zinciridir. İspatta kullanılan **çıkarım kurallarının (inference rules)** en tanınmışı **Modus Ponens**tir:

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta} \quad \text{(Modus Ponens).}$$

Örneğin (`WumpusAhead ∧ WumpusAlive`) ⇒ `Shoot` kuralı ile (`WumpusAhead ∧ WumpusAlive`) bilgisi varsa `Shoot` çıkarılır. Başka yararlı kural **And-Elimination**dır:

$$\frac{\alpha \wedge \beta}{\alpha},$$

"Önümde Wumpus var VE yaşıyor" bilgisinden "Wumpus yaşıyor" elde edilir. Bu kuralların sağlamlığı doğruluk tablosundan bir kez gösterilebilir; sonrasında her uygulamada bütün modelleri incelemek gerekmez. Şekil 7.11'deki eşdeğerlikler de çıkarım kuralı olarak kullanılabilir. Örneğin $\alpha \Leftrightarrow \beta$ çift koşulu iki tek yönlü koşulun VE'sine çevrilir ve gerekirse tersine çevrilir. Buna karşılık **Modus Ponens ters işletilemez**: β'dan tek başına α ve $\alpha \Rightarrow \beta$ elde edilemez.

Wumpus KB'sinde [1,2]'nin çukursuz olduğuna ispat

`R1...R5` bilgi tabanından $\neg P_{1,2}$ göstermek istiyoruz. Adımlar:

1. **R2:** $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$.
2. **R6:** Çift koşulu kaldır: $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$.
3. **R7:** VE'yi kaldır: $(P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}$.
4. **R8:** Karşıt tersi al: $\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})$.
5. **R9:** R4 $\neg B_{1,1}$ ile Modus Ponens uygula: $\neg(P_{1,2} \vee P_{2,1})$.
6. **R10:** De Morgan uygula: $\neg P_{1,2} \wedge \neg P_{2,1}$.

Böylece [1,2] ve [2,1]'in ikisinin de çukur içermediği ispatlanır. Dikkat: Bu ispat $B_{2,1}, P_{1,1}, P_{2,2}, P_{3,1}$ kullanmaz; bunlar hedef için ilgisizdir.

İspat arama problemi olarak nasıl tanımlanır?

3. bölümün arama algoritmaları bir ispatın adımlarını da arayabilir: **Başlangıç durumu** mevcut KB, **eylemler** uygulanabilir çıkarım kuralları, **sonuç** türetilmiş cümlenin KB'ye eklenmesi, **hedef** ispatlanacak cümlenin yer aldığı durumdur. Model sayımı yerine kısa ispat aramak ilgisiz milyonlarca bilgi bulunduğunda avantaj sağlayabilir.

Monotonluk (Monotonicity)

Klasik mantık **monotondur**: yeni bilgi eklemek daha önce gerektirilen sonucu geri almaz.

$$\boxed{KB \models \alpha \implies KB \wedge \beta \models \alpha.}$$

Örneğin dünyada tam sekiz çukur bulunduğu bilgisi daha yeni sonuçlar sağlayabilir; ancak zaten kanıtlanmış "[1,2]'de çukur yok" sonucunu geçersiz kılamaz. Klasik mantığın aksine **nonmonotonic** mantıklar, insanın yeni bilgiyle fikrini değiştirmesini daha doğrudan ele alır; 12.6. bölümde anlatılır.

7.5.2 — Resolution ile İspat

İlk kurallarımız sound olsa da hepsi birden yeterli olmadığında complete bir ispat sistemi oluşmayabilir. Örneğin çift koşulu kaldırma kuralını çıkarsaydık az önceki ispatı aynı kurallarla tamamlayamazdık. **Resolution (çözümleme)**, uygun tam aramayla birleştiğinde önermeler mantığı için tam bir çıkarım yöntemi sunan tek bir çıkarım kuralıdır.

Wumpus dünyasında ilk resolution adımları

Ajan [1,2]'de koku algılar ama esinti algılamaz. KB'ye:

$$R_{11} : \neg B_{1,2},$$
$$R_{12} : B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$$

eklenir. Önceki benzer çıkarımlarla $R_{13} : \neg P_{2,2}$ ve $R_{14} : \neg P_{1,3}$ bulunur (zaten $\neg P_{1,1}$ bilinir). R3'teki çift koşulu kaldırıp R5'i uygularsak:

$$R_{15} : P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

elde edilir. $P_{2,2}$ ile $\neg P_{2,2}$ birbirini tamamlayan literallerdir; bunları eleyerek:

$$\frac{P_{1,1} \vee P_{2,2} \vee P_{3,1}, \quad \neg P_{2,2}}{P_{1,1} \vee P_{3,1}} \quad (R_{16})$$

buluruz. Ardından R1 $\neg P_{1,1}$ ile tekrar çözümleme:

$$\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1}}{P_{3,1}} \quad (R_{17}).$$

Yani [3,1]'de kesin çukur vardır. Bu iki adım **unit resolution** örneğidir.

Clause, unit clause, tamamlayıcı literal ve resolvent

Clause, literallerin VEYA'sıdır. $P \vee \neg Q \vee R$ bir clause; P ise tek literalden oluşan **unit clause**'tur. P ve $\neg P$ **tamamlayıcı (complementary)** literallerdir. Resolution'ın ürettiği yeni clause'a **resolvent** denir.

Genel kural, iki clause'daki birbirinin tamamlayıcısı birer literali çıkarır ve kalanları VEYA ile birleştirir. Örneğin:

$$\frac{P_{1,1} \vee P_{3,1}, \qquad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}.$$

Tekrarlanan literal tek kopyaya düşürülür (**factoring**): `(A ∨ B)` ile `(A ∨ ¬B)` → `(A ∨ A)` → `A`. Bir literal doğruysa onu taşıyan clause doğru; yanlışsa o clause'un kalan kısmı doğru olmak zorundadır; bu iki olasılık Resolution'ın sound olmasını sağlar.

Bağlaçsal normal biçim (CNF)

Resolution yalnızca clause'lara uygulanır. Yine de önermeler mantığının **her cümlesi**, clause'ların VE ile birleşimine eşdeğer biçime çevrilebilir. Bu biçim **CNF (conjunctive normal**

form)tur.

Şu Wumpus kuralını CNF'ye çevirelim:

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

1 — $\Leftrightarrow$ kaldır:

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}).$$

2 — $\Rightarrow$ kaldır ($\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$):

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}).$$

3 — Olumsuzlamayı içeri taşı (De Morgan ve çift olumsuzlama):

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}).$$

4 — VEYA'yı VE üzerine dağıt:

$$\boxed{\begin{array}{l}(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \\ \wedge\, (\neg P_{1,2} \vee B_{1,1}) \\ \wedge\, (\neg P_{2,1} \vee B_{1,1}).\end{array}}$$

Artık üç clause'un VE'si elde edilmiştir. Özgün cümleden daha zor okunur ama Resolution algoritmasına doğrudan verilebilir.

Şekil 7.12 — PL-RESOLUTION

Çelişkiyle ispat için $KB \models \alpha$ yerine $KB \wedge \neg\alpha$'nın doyurulamazlığı sınanır. Önce CNF'ye çevir, tamamlayıcı literal içeren bütün clause çiftlerinden resolvent üret, yenilerini kümeye ekle. Yeni clause üretilemiyorsa gerektirme yoktur. **Boş clause** □ bulunursa çelişki vardır ve gerektirme doğrudur.

```
function PL-RESOLUTION(KB, alpha) -> true/false
  clauses <- CNF(KB ∧ ¬alpha) içindeki clause kümesi
  new <- {}
  loop:
    for each Ci, Cj in clauses:
      resolvents <- PL-RESOLVE(Ci, Cj)
      if boş clause ∈ resolvents:
        return true
      new <- new ∪ resolvents
    if new ⊆ clauses:
      return false
    clauses <- clauses ∪ new
```

Bir boş clause, hiç literal içermeyen VEYA'dır ve `False` ile eşdeğerdir. Özellikle tekli iki clause P ve ¬P birbirine çözümlenince □ oluşur.

Şekil 7.13: Resolution ispat ağacı

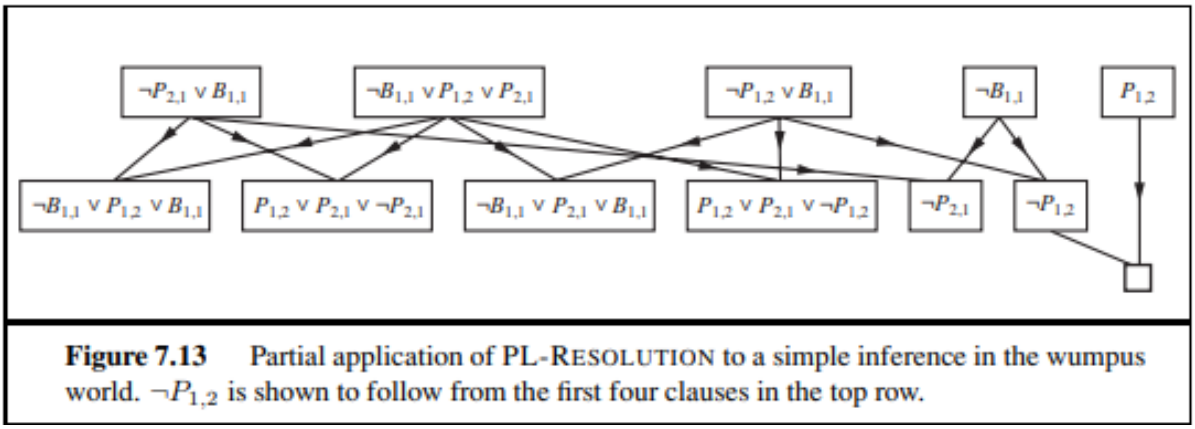


Figure 7.13 Partial application of PL-RESOLUTION to a simple inference in the wumpus world. $\neg P_{1,2}$ is shown to follow from the first four clauses in the top row.

Şekil 7.13 — Wumpus'un başlangıç karesinde esinti olmayışından $\neg P_{1,2}$ çıkışı. Son satırdaki küçük kare boş clause'u, yani çelişkiyi gösterir.

İlgili KB: $R_2 \wedge R_4$; ispatlanacak $\alpha = \neg P_{1,2}$. Tersini ekleriz: $\neg\alpha = P_{1,2}$. CNF clause'ları:

$$C_1 : \neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$$
$$C_2 : \neg P_{1,2} \vee B_{1,1}$$
$$C_3 : \neg P_{2,1} \vee B_{1,1}$$
$$C_4 : \neg B_{1,1}$$
$$C_5 : P_{1,2}.$$

C2 ile C4 çözümlemesinden $\neg P_{1,2}$ çıkar. Bunu C5 ile çözümleyince **boş clause** elde edilir. Çelişki, $KB \models \neg P_{1,2}$ demektir. Şekildeki bazı yollar $B_{1,1} \vee \neg B_{1,1} \vee P_{1,2}$ gibi zaten `True` olan clause'lar üretir; bu tür **totolojik clause**'lar atılarak gereksiz iş azaltılır.

Resolution'ın tamlığı: kapanış ve teorem

Bir S clause kümesinin **resolution kapanışı**, $RC(S)$, S'deki clause'lardan tekrar tekrar Resolution kullanılarak türetilebilen bütün clause'ların kümesidir. Kaynak semboller sonlu ve factoring kullanıldığında farklı clause sayısı da sonludur; bu nedenle `PL-RESOLUTION` sonlanır.

Ground Resolution Theorem: Bir clause kümesi doyurulamazsa kapanışında boş clause vardır:

$$S \text{ doyurulamaz } \Rightarrow \ \square \in RC(S).$$

İspat, karşıt ters önerme üzerinden yapılır: $\square \notin RC(S)$ ise S için bir model kurulabilir. $P_1, ..., P_k$ sembolleri sırayla atanır. $\neg P_i$ içeren ve diğer literalleri önceki atamalarla yanlış olmuş bir clause varsa $P_i = False$; aksi hâlde $P_i = True$ seçilir. Seçim sonucu ilk yanlış clause'un oluştuğunu varsayarsak, aynı anda hem P_i hem $\neg P_i$ biçimindeki tamamlayıcı gereksinimler ortaya çıkmak zorundadır. Bunların resolvent'i ise daha önce yanlış olmuş bir clause olur ve "ilk yanlışlık i. adımda oluştu" varsayımıyla çelişir. Böylece atama S'nin modelidir ve teorem tamamlanır.

Bu tamlık sonucu şaşırtıcıdır: **Tek Resolution kuralı**, sistematik uygulandığında önermeler mantığında doyurulamazlığı tespit etmeye yeter.

7.5.3 — Horn Clause'lar ve Kesin Clause'lar

Her bilgi tabanının genel Resolution'a ihtiyacı yoktur. Belirli biçimdeki cümlelere sahip sistemlerde daha kısıtlı ve daha hızlı yöntemler kullanılabilir.

Definite clause (kesin clause), **tam bir** pozitif literal içerir. Örneğin $\neg L_{1,1} \vee \neg Breeze \vee B_{1,1}$ definite clause'tur. Fakat $\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$ iki pozitif literal içerdiğinden değildir.

Horn clause, **en fazla bir** pozitif literal içerir. Her definite clause bir Horn clause'tur. Hiç pozitif literal içermeyen Horn clause ise **goal clause** olarak adlandırılır. Horn clause'lar Resolution altında kapalıdır: ikisinin resolvent'i de Horn olur.

Şekil 7.14: CNF, Horn ve definite dilbilgisi

$$
\begin{aligned}
\mathit{CNFSentence} &\rightarrow \mathit{Clause}_1 \wedge \cdots \wedge \mathit{Clause}_n \\
\mathit{Clause} &\rightarrow \mathit{Literal}_1 \vee \cdots \vee \mathit{Literal}_m \\
\mathit{Literal} &\rightarrow \mathit{Symbol} \mid \neg \mathit{Symbol} \\
\mathit{Symbol} &\rightarrow P \mid Q \mid R \mid \ldots \\
\mathit{HornClauseForm} &\rightarrow \mathit{DefiniteClauseForm} \mid \mathit{GoalClauseForm} \\
\mathit{DefiniteClauseForm} &\rightarrow (\mathit{Symbol}_1 \wedge \cdots \wedge \mathit{Symbol}_l) \Rightarrow \mathit{Symbol} \\
\mathit{GoalClauseForm} &\rightarrow (\mathit{Symbol}_1 \wedge \cdots \wedge \mathit{Symbol}_l) \Rightarrow \mathit{False}
\end{aligned}
$$

Figure 7.14 A grammar for conjunctive normal form, Horn clauses, and definite clauses. A clause such as $A \wedge B \Rightarrow C$ is still a definite clause when it is written as $\neg A \vee \neg B \vee C$, but only the former is considered the canonical form for definite clauses. One more class is the k-CNF sentence, which is a CNF sentence where each clause has at most k literals.

Şekil 7.14 — CNF, Horn ve definite clause biçimlerinin dilbilgisi. $A \wedge B \Rightarrow C$, $\neg A \vee \neg B \vee C$ ile eşdeğerdir; definite clause'un alışılmış (canonical) yazımı soldakidir.

```
CNFSentence       -> Clause1 ∧ ... ∧ Clausen
Clause            -> Literal1 ∨ ... ∨ Literalm
Literal           -> Symbol | ¬Symbol
Symbol            -> P | Q | R | ...
HornClauseForm    -> DefiniteClauseForm | GoalClauseForm
DefiniteClauseForm-> (Symbol1 ∧ ... ∧ Symboll) ⇒ Symbol
GoalClauseForm    -> (Symbol1 ∧ ... ∧ Symboll) ⇒ False
```

k-CNF, her clause'unda en fazla k literal olan CNF'dir. Örneğin $(P \vee Q) \wedge (\neg P \vee R \vee S)$ bir 3-CNF cümlesidir.

Definite clause'ların üç yararı

1. Her biri, **pozitif öncüllerin VE'si ⇒ tek pozitif sonuç** biçiminde yazılabilir: $\neg L_{1,1} \vee \neg Breeze \vee B_{1,1}$ yerine $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$. Öncüle **body**, sonuca **head** denir. Yalnızca $L_{1,1}$ gibi pozitif literale **fact (olgu)** denir; $True \Rightarrow L_{1,1}$ diye de düşünülebilir.
2. Bu KB'lerde **forward chaining** ve **backward chaining** insanın da kolay takip ettiği doğal çıkarım adımları sunar; bunlar mantıksal programlamanın temelidir (9. bölüm).

3. Definite/Horn yapıdaki uygun çıkarım problemleri, KB büyüklüğüyle **doğrusal zamanda** çözülebilir; genel SAT problemlerinin karmaşıklığına göre büyük avantajdır.

7.5.4 — İleri ve Geri Zincirleme

Forward chaining: veri odaklı çıkarım

PL-FC-ENTAILS?(KB,q) definite clause'lar üzerinde hedef atomik q'nun KB'den çıkıp çıkmadığını belirler. **Bilinen fact'lerden** başlar. Bir kuralın bütün öncülleri doğrulanınca sonucunu fact kümesine ekler; q bulunana veya başka çıkarım yapılamayana kadar sürer. Örneğin $L_{1,1}$ ve $Breeze$ biliniyor, $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$ kuralı varsa $B_{1,1}$ çıkar.

Şekil 7.15: Forward chaining sözde kodu

```
function PL-FC-ENTAILS?(KB, q)
  count[c] <- c kuralındaki henüz sağlanmamış öncül sayısı
  inferred[s] <- false  // her sembol için
  agenda <- KB'de başta doğru olduğu bilinen fact kuyruğu
  while agenda boş değil:
    p <- POP(agenda)
    if p = q: return true
    if inferred[p] = false:
      inferred[p] <- true
      for c in KB, p ∈ c.PREMISE:
        count[c] <- count[c] - 1
        if count[c] = 0:
          agenda'ya c.CONCLUSION ekle
  return false
```

agenda, bilinen ama henüz işlenmemiş sembolleri; count, kuralın kalan öncül sayısını takip eder. Daha önce işlenmiş sembolün yeniden işlenmemesi, P ⇒ Q, Q ⇒ P gibi döngülerde tekrarları önler. Uygun indekslemeyle her öncül ilişkisi yalnızca gerekli sayıda işlenir ve toplam süre KB boyutunda doğrusaldır.

Şekil 7.16: Horn kuralları ve AND–OR grafı

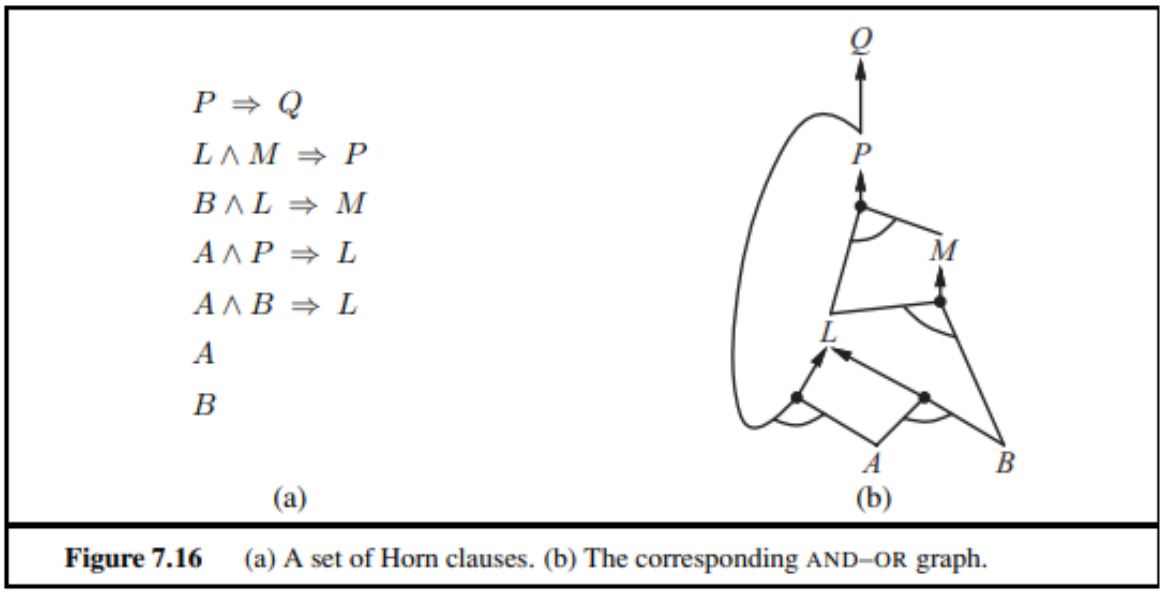


Figure 7.16 (a) A set of Horn clauses. (b) The corresponding AND–OR graph.

Şekil 7.16 — (a) Bilgi tabanındaki Horn kuralları ve başlangıç olguları. (b) Aynı kuralların AND–OR grafı. Yayla birleştirilmiş kolların hepsi (AND) kanıtlanmalı; bağımsız alternatiflerden biri (OR) yeterlidir.

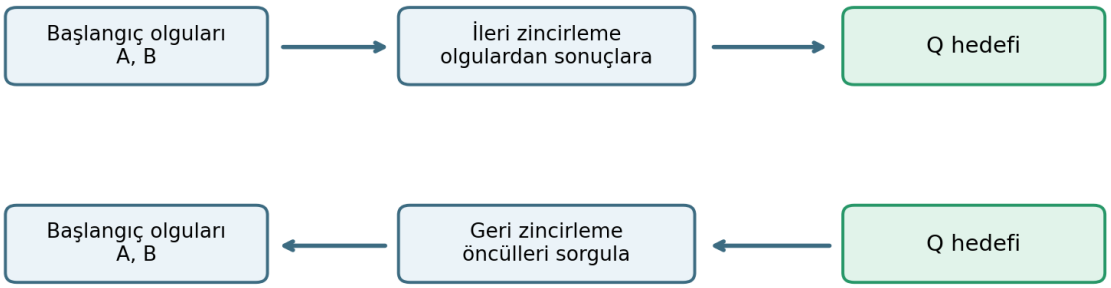
Şekil 7.16(a)'daki bilgi tabanı:

$$P \Rightarrow Q, \quad L \wedge M \Rightarrow P, \quad B \wedge L \Rightarrow M,$$
$$A \wedge P \Rightarrow L, \quad A \wedge B \Rightarrow L, \quad A, \; B.$$

A ve B başlangıç olgularıdır. İleri zincirlemenin doğal yolu:

$$A, B \Rightarrow L \Rightarrow M \Rightarrow P \Rightarrow Q.$$

Burada her ok yalnızca yeni elde edilen bir olguyu özetler; örneğin M'yi elde etmek için B ile L'nin **ikisi de** gereklidir. Şekil 7.16(b)'de ilişkilerin AND/OR biçiminde gösterilmesinin sebebi budur.



Öğretici ek şekil — İleri zincirleme olgulardan hedefe; geri zincirleme hedeften olgulara doğru gerekçeleri araştırır.

Forward chaining neden sağlam ve tamdır?

Her çıkarım Modus Ponens uygulaması olduğundan **sound**dur. Tamlık için algoritmanın artık yeni bilgi üretemediği **sabit noktaya (fixed point)** bakalım. Bu noktada `inferred` tablosunda

bulunan her sembol doğru, diğerleri yanlış alınarak bir model kurulur. Herhangi bir definite kural $a_1 \wedge \ldots \wedge a_k \Rightarrow b$ bu modelde yanlış olsaydı öncülleri doğru, b yanlış olmalıydı. Ama bütün öncüller bulunuyorsa algoritma b'yi zaten türetir; bu sabit noktayla çelişir. Böylece model KB'yi sağlar. KB'nin gerektirdiği her atomik q, bu modelde de doğru olacağı için ileri zincirleme tarafından bulunmuş olmak zorundadır.

Bu yöntem **veri odaklı çıkarım (data-driven reasoning)** örneğidir; yeni algı geldiğinde belirli bir sorgu olmadan yeni sonuçlar üretilebilir. Yağmurun başladığını duyunca pikniğin iptal olabileceğini düşünmemiz gibi. İnsanlar ilgisiz sonuçların altında kalmamak için bu çıkarımları sınırlar; örneğin komşunun bahçesindeki en büyük gülün on yedinci yaprağının ıslanmasını düşünmemiz gerekmez.

Backward chaining: hedef odaklı çıkarım

Geri zincirleme, **sorgudan geriye** çalışır. q zaten fact ise biter. Değilse sonucunda q olan kuralları arar; en az bir kuralın tüm öncüllerini yine geri zincirlemeyle ispatlayabilirse q'yu ispatlar. Şekil 7.16'da Q için "Q olması için P; P için L ve M; L için A ve B; M için B ve L" diye geriye gidilir. Sonunda A ve B olgularına ulaşılır. Bu yaklaşım 4. bölümdeki AND–OR-GRAPH-SEARCH'e benzer.

Ders çözümü (Q hedefi):

1. Q için P ⇒ Q: P'yi kanıtla.
2. P için L ∧ M ⇒ P: L ve M'yi kanıtla.
3. L için A ∧ B ⇒ L: A ve B fact, o hâlde L doğru.
4. M için B ∧ L ⇒ M: B ve L doğru, o hâlde M doğru.
5. L ve M'den P, P'den Q doğru.

Verimli backward chaining uygulaması da doğrusal olabilir; üstelik yalnızca ilgili kuralları ziyaret ederse pratik maliyeti KB boyutundan çok küçük kalabilir. "Şimdi ne yapmalıyım?" veya "Anahtarım nerede?" gibi belirli sorularda hedef odaklı düşünmek doğaldır.

Karşılaştırma	İleri zincirleme	Geri zincirleme
İlk adım	Var olan fact'ler	Sorgulanan hedef
Akış	Veriden sonuçlara	Sonuçtan gerekli öncüllere
Terim	Data-driven	Goal-directed
Uygun örnek	Yeni sensör algılarından yeni bilgiler	Belirli bir soruyu ispatlama
Risk	Çok sayıda ilgisiz sonuç üretimi	Öncüllerde döngü/tekrar; uygun bellekleme gerekir

7.6 — Etkili Önermeler Mantığı Model Denetimi

Genel önermesel çıkarım için iki etkili model denetimi yaklaşımını inceleyeceğiz: **geri izlemeli arama** ve **yerel tepe tırmanma araması**. Bunlar önermeler mantığının uygulanabilmesini sağlayan teknolojinin parçasıdır. Kitap bu teknik bölümün ilk okumada hızlıca gözden geçirilebileceğini belirtir.

Algoritmalar doğrudan **SAT problemini**, yani bir cümlenin doyurulabilir olup olmadığını inceler. Gerekirme, $\alpha \models \beta$ yerine $\alpha \wedge \neg\beta$'nın doyurulamazlığı sınanarak ele alınabilir. 6. bölümdeki kısıt sağlama problemleriyle akrabalıkları doğaldır: ikisinde de uygun değişken ataması aranır. Pek çok kombinatoryal problem SAT'a indirgenebildiğinden SAT çözücülerin gelişmesi çok geniş uygulama alanlarına yansır.

7.6.1 — Tam bir geri izleme algoritması: DPLL

İlk yöntem sıklıkla Davis–Putnam algoritması diye anılır (Davis ve Putnam, 1960). Buradaki sürüm Davis, Logemann ve Loveland'ın (1962) tarifidir; dört soyadın baş harflerinden **DPLL** denir. DPLL, **CNF biçiminde cümle**, yani clause kümesi alır. TT-ENTAILS? gibi olası modellere derinlik öncelikli, özyinelemeli şekilde bakar; ancak üç önemli iyileştirme kullanır.

İyileştirme 1: Erken sonlandırma (Early termination)

Kısmi model, sonucun kesinleşmesi için yeterli olabilir. Örneğin $(A \vee B) \wedge (A \vee C)$ için yalnız $A = True$ ataması bütün cümleyi doğru yapar; B ve C'yi incelemek gerekmez. Tersine, herhangi bir clause'un bütün literalleri yanlışsa CNF'nin tamamı yanlıştır ve o arama dalı bırakılır. Böylece arama ağacının tüm alt dalları budanabilir.

İyileştirme 2: Saf sembol sezgiseli (Pure-symbol heuristic)

Saf sembol, henüz dikkate alınması gereken clause'larda yalnızca tek işaretle görünendir. Örnek üç clause:

$$(A \vee \neg B), \qquad (\neg B \vee \neg C), \qquad (C \vee A).$$

A yalnız pozitif, B yalnız negatif olarak görünür: **saf**tır. C hem C hem ¬C biçimindedir: saf değildir. Saf sembolü, göründüğü literali doğru yapacak biçimde atamak hiçbir clause'u yanlış hâle getirmez; en az bir model aranırken bu güvenle kullanılabilir. Örneğin `B=False` olduğunda `(¬B ∨ ¬C)` zaten doğrudur; bu clause dikkate alınmazsa geriye kalanlarda C yalnızca pozitif görünebilir, yani **C sonradan saflaşabilir**.

Öğretici tablo:

Sembol	Görünüm	Saf mı?	Yararlı atama
A	A, A	Evet	A = True

Sembol	Görünüm	Saf mı?	Yararlı atama
B	¬B, ¬B	Evet	B = False
C	C, ¬C	Hayır	Henüz zorunlu değil

İyileştirme 3: Birim clause sezgiseli (Unit-clause heuristic)

Birim clause yalnız bir literal içerir. DPLL bağlamında, diğer bütün literalleri **atanıp yanlış yapılmış**, geriye tek atanmamış literal kalmış clause da **unit** sayılır. Örneğin B=True ise (¬B ∨ ¬C) → ¬C; bu clause'un doğru olması için C=False zorunludur. Birim clause'lara göre zorunlu atamalar, iki dalda arama başlatmadan önce yapılır.

Bu atama yeni unit clause üretebilir: C=False olduğunda (C ∨ A) → A, dolayısıyla A=True. Ardışık zorunlu atamalara **unit propagation (birim yayılım)** denir. Definite clause'lardaki forward chaining'e benzer; yalnız definite clause içeren bir CNF üzerinde DPLL esasen ileri zincirlemeyi yeniden üretir. Zaten KB'de bulunan bir literal için çelişkiyle ispat açıldığında da süreç çok çabuk tamamlanabilir.

Şekil 7.17 — DPLL sözde kodu

```
function DPLL-SATISFIABLE?(s) -> true/false
  clauses <- s ifadesinin CNF clause'ları
  symbols <- s içindeki önerme sembolleri
  return DPLL(clauses, symbols, {})

function DPLL(clauses, symbols, model)
  if bütün clause'lar modelde doğru: return true
  if bir clause modelde yanlış: return false

  P, value <- FIND-PURE-SYMBOL(symbols, clauses, model)
  if P bulunduysa:
    return DPLL(clauses, symbols - P, model U {P=value})

  P, value <- FIND-UNIT-CLAUSE(clauses, model)
  if P bulunduysa:
    return DPLL(clauses, symbols - P, model U {P=value})

  P <- FIRST(symbols)
  rest <- REST(symbols)
  return DPLL(clauses, rest, model U {P=true})
      or DPLL(clauses, rest, model U {P=false})
```

DPLL bir **tam SAT karar yöntemi**dir: varsa doyurucu atamayı bulabilir; yoksa aramayı sonlandırıp yokluğunu belirler. Önce kısmi modelde kesin karar verilip verilemeyeceği sınanır; saf sembol ve unit clause denenir; sonunda gerekirse true/false dallanmasına geçilir.

Büyük SAT çözücüleri hangi tekniklerle hızlanır?

Şekil 7.17 yalnızca iskeleti gösterir. Modern çözücüler, daha önce gördüğümüz genel arama stratejilerinin ileri biçimlerini kullanır:

1. **Bileşen analizi (component analysis).** Kısmi atamalardan sonra ortak atanmamış değişken paylaşmayan bağımsız clause grupları oluşabilir. Bunları (6. bölümdeki Tazmanya/CSP örneğindeki gibi) ayrı çözmek hesaplamayı azaltır.
2. **Değişken ve değer sırası.** Basit DPLL ilk sembolü ve önce `true` değerini seçer; derece sezgiseli (degree heuristic) kalan clause'larda en çok görünen sembolün seçilmesini önerir.
3. **Akıllı geri izleme.** Hata oluşunca yalnız bir adım geri gitmek yerine çatışmaya neden olan önceki karara kadar sıçramak büyük kazanç sağlayabilir. **Conflict clause learning**, aynı çatışmayı tekrar yaşamamak için yeni kısıt cümleleri saklar; nadiren kullanılanlar silinebilir.
4. **Rastgele yeniden başlatma.** Arama ilerlemiyorsa farklı seçimlerle ağacın tepesinden başlanabilir; öğrenilen clause'lar korunur. Hızı her örnekte garanti etmez ama çözüm süresinin değişkenliğini azaltabilir.
5. **Akıllı indeksleme.** Bir sembolün hangi clause'larda pozitif/negatif geçtiğini hızlıca bulmak gerekir. Artık sağlanmış clause'lar dışlandığından bu dizinlerin hesap sırasında güncellenmesi gerekir.

Bu geliştirmeler modern çözücülerin on milyonlarca değişkeni işleyebilmesini sağlar; donanım ve güvenlik protokolü doğrulaması gibi alanlarda eskiden insan yönlendirmesine dayanan yoğun ispat süreçlerini büyük ölçüde değiştirmiştir.

7.6.2 — Yerel arama ve WalkSAT

Hill-climbing ve simulated annealing türü yerel arama yöntemleri, uygun değerlendirme fonksiyonuyla SAT'a uygulanabilir. Her clause'u doğru yapmayı hedeflediğimiz için **yanlış clause sayısı** iyi bir maliyet fonksiyonudur; 6. bölümdeki MIN-CONFLICTS de benzer sayımı kullanır. Bu algoritmalar **tam değer atamaları** üzerinde tek bir sembolün doğruluk değerini tersine çevirerek hareket eder. Yerel minimumlardan kurtulmak için rastgelelik gerekir; açgözlülük ile rastgeleliğin dengesi önemlidir.

WalkSAT, her yinelemede yanlış bir clause seçer ve içindeki bir sembolü değiştirir. Sembol seçimi rastgele iki yoldan yapılır: (1) yeni durumda yanlış clause sayısını en aza indiren **min-conflicts** hareketi; (2) o clause içinden rastgele sembol seçen **random walk** hareketi.

Şekil 7.18 — WalkSAT sözde kodu

```
function WALKSAT(clauses, p, max_flips)
  model <- sembollere rastgele true/false ata
  for i = 1 to max_flips:
    if model tüm clause'ları sağlıyorsa:
      return model
    clause <- yanlış clause'lardan birini rastgele seç
    with probability p:
      clause'daki rastgele sembolün değerini ters çevir
    otherwise:
      en çok clause'u doğru yapan sembolü ters çevir
  return failure
```

p rastgele yürüyüş olasılığıdır (kitapta tipik yaklaşık 0,5); `max_flips` izin verilen değişiklik sayısıdır. WalkSAT model bulursa formül **kesin doyurulabilir**dir. Fakat `failure`, "çözüm yok" veya "yeterince aramadık" anlamlarından hangisinin doğru olduğunu söylemez. `max_flips=∞` ve `p>0` iken **var olan çözümün eninde sonunda bulunması olasılık 1'dir**, fakat doyurulamaz formülde algoritma hiç sonlanmayabilir. Bu nedenle ispat gerektiren Wumpus güvenliği sorguları için tek başına güvenilir değildir: "tehlikeli bir model bulamadım" demek "tehlike imkânsız" demek değildir.

Çözümlü WalkSAT örneği:

$$(A \vee B) \wedge (\neg A \vee C) \wedge (B \vee C)$$

Başlangıç `A=B=C=False` ise 1. ve 3. clause yanlıştır; maliyet 2. Yanlış ilk clause seçilip B çevrilirse `B=True` olur ve tüm clause'lar doğru hâle gelir; maliyet 0. Bu özel örnekte tek flip yeterlidir; genel durumda arama birçok adım gerektirebilir.

7.6.3 — Rastgele SAT Problemlerinin Görünümü

SAT örneklerinin zorlukları farklıdır. N-vezir probleminde olduğu gibi bazı kısıt problemleri, pek çok çözümün yakın bulunması sayesinde yerel aramayla kolay olabilir; bu **az kısıtlılık (under-constrained)** olgusudur. CNF'de az clause, çok sayıda uygun atama demek olabilir. Aşağıdaki beş sembollü, beş clause'lu rastgele 3-CNF örneği verilir:

$$(\neg D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C)$$
$$\wedge (\neg C \vee \neg B \vee E) \wedge (E \vee \neg D \vee B)$$
$$\wedge (B \vee E \vee \neg C).$$

Beş sembol $2^5 = 32$ atama oluşturur; **16'sı** bu cümleyi sağlar. Dolayısıyla bağımsız rastgele tahminde başarı olasılığı 1/2, ilk başarıya kadar beklenen deneme sayısı 2'dir. Çok kısıtlı (over-constrained) örneklerde clause sayısı değişken sayısına göre fazladır ve çoğunlukla çözüm yoktur.

Rastgele cümle üretimini belirginleştirmek için $CNF_k(m, n)$ gösterimi kullanılır: **m** clause, **n** sembol, her clause'da **k** farklı literal; semboller ve işaretleri rastgele seçilir; aynı sembol bir clause'da iki kez ve aynı clause bir cümlede iki kez yer almaz.

Şekil 7.19 — Olasılık ve çalışma maliyeti grafikleri

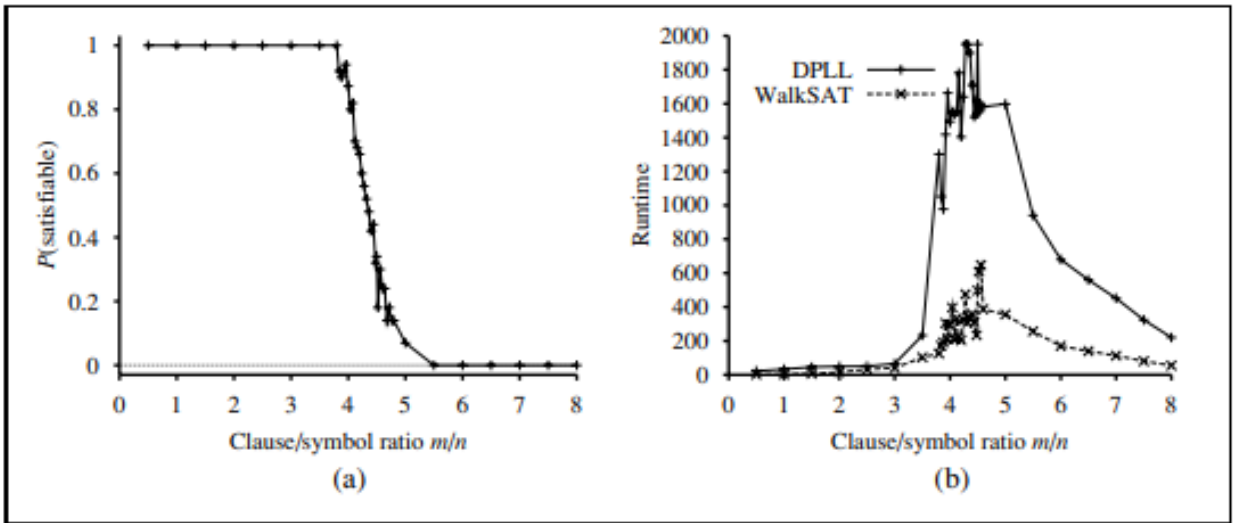


Figure 7.19 (a) Graph showing the probability that a random 3-CNF sentence with $n = 50$ symbols is satisfiable, as a function of the clause/symbol ratio m/n. (b) Graph of the median run time (measured in number of recursive calls to DPLL, a good proxy) on random 3-CNF sentences. The most difficult problems have a clause/symbol ratio of about 4.3.

Şekil 7.19 — (a) n=50 sembollü rastgele 3-CNF formüllerinin doyurulabilir olma olasılığı; (b) DPLL

ve WalkSAT'in medyan çalışma maliyeti. İki grafiğin yatay ekseni clause/sembol oranı m/n'dir.

Şekil 7.19(a)'da m/n küçükken doyurulabilirlik olasılığı 1'e, büyükken 0'a yaklaşır. **Yaklaşık 4,3** oranında keskin düşüş vardır. Örneğin n=50 için bu oran yaklaşık m=50×4,3=215 clause demektir. Kitabın aktardığı **satisfiability threshold conjecture**, $k \geq 3$ için r_k adlı kritik oran olduğu ve n→∞ iken altında olasılığın 1'e, üstünde 0'a gittiği önerisidir; kitabın aktardığı aşamada genel varsayım ispatlanmış değildir.

Şekil 7.19(b)'de **en zor problemler de çoğu zaman aynı eşik bölgesindedir**. Yaklaşık 4,3 oranındaki 50 sembollü problemler, 3,3 oranındakilere göre yaklaşık 20 kat daha zor olabilir. Az kısıtlı problemler uygun model kolay bulunduğu için; çok kısıtlı problemler ise uyuşmazlıklar görece çabuk ortaya çıkabildiği için daha kolay olabilir. Bu grafik belirli bir rastgele problem ailesinin **deneysel** davranışıdır; her tekil SAT örneği için evrensel zaman garantisi değildir.

Öğretici yorum: "Daha çok kısıt = daima daha zor" yanlıştır. En zor bölge çoğu zaman "çözüm var mı yok mu?" ayrımının sınırıdır.

7.7 — Önermeler Mantığına Dayalı Ajanlar

Şimdi bütün konuları bir araya getirerek Wumpus dünyasında önermeler mantığı kullanan ajanlar tasarlıyoruz. İlk hedef, **algı geçmişinden mevcut dünya durumunu mümkün olduğunca çıkarabilmek**tir. Bunun için eylemlerin etkilerini de içeren mantıksal model gerekir. İkinci hedef, her soruda geçmişi baştan analiz etmeden güncel durumu verimli takip etmektir. Son aşamada mantıksal çıkarımla hedefe ulaşan **planlar** elde edeceğiz.

7.7.1 — Dünyanın mevcut durumu (The current state)

Mantıksal ajan, dünya hakkındaki cümleler KB'sinden ne yapacağını çıkarır. KB, (1) dünyanın işleyişine ilişkin genel aksiyomlar ve (2) özel bir dünyada ajan tarafından algılanmış cümlelerden oluşur. "Neredeyim?", "Bu kare güvenli mi?" gibi soruların yanıtları hedeflenir.

Çukur, Wumpus, koku ve esinti kuralları

Başlangıçta ne çukur ne Wumpus vardır:

$$\neg P_{1,1}, \qquad \neg W_{1,1}.$$

Her kare için "komşuda çukur iff esinti" ve "komşuda Wumpus iff koku" kuralları bulunur. Örneğin:

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}), \qquad S_{1,1} \Leftrightarrow (W_{1,2} \vee W_{2,1}).$$

Ajan **tam olarak bir Wumpus** olduğunu da bilir. Bu iki ayrı iddiayı gerektirir:

- **En az bir Wumpus:** $W_{1,1} \vee W_{1,2} \vee \cdots \vee W_{4,3} \vee W_{4,4}$.
- **En fazla bir Wumpus:** Her farklı kare çifti için $\neg W_{a,b} \vee \neg W_{c,d}$ (örneğin $\neg W_{1,1} \vee \neg W_{1,2}$ ve $\neg W_{4,3} \vee \neg W_{4,4}$). Böylece iki farklı karede aynı anda Wumpus bulunamaz.

Ders örneği: "En az bir" kuralı iki Wumpus'u yasaklamaz. "En fazla bir" kuralı ise sıfır Wumpus'a izin verir. İkisi birlikte "tam olarak bir" sonucunu sağlar.

Algı cümlelerinde zaman indeksi zorunluluğu

Bir zamanda koku yokken başka bir zamanda koku olabilir. KB'ye zamansız `Stench` ile `¬Stench` eklersek çelişki yaratırız. Bu yüzden algıyı **zamana bağlarız**: $Stench^4$ zaman 4'te koku, $\neg Stench^3$ zaman 3'te koku yok anlamına gelir; çelişmezler. Aynı ilke `Breeze`, `Bump`, `Glitter`, `Scream` için uygulanır. Şekil 7.1'deki `MAKE-PERCEPT-SENTENCE` zaman argümanını bu yüzden alır.

Fluent: zamanla değişebilen durum değişkenleri

Zamanla değişebilen dünya özelliklerine **fluent (durum değişkeni)** denir; `fluens` Latince "akan" demektir. Başlangıç bilgileri şunları içerebilir:

$$L^0_{1,1}, \qquad FacingEast^0, \qquad HaveArrow^0, \qquad WumpusAlive^0.$$

Burada ajan zaman 0'da [1,1]'dedir, doğuya bakar, oku vardır, Wumpus hayattadır. Çukur konumu gibi zamanla değişmeyen özellikler **atemporal variables (zamandan bağımsız değişkenler)** olarak yazılır; üst zaman indeksi taşımaz. Örneğin $B_{x,y}$ bir karenin sabit esinti özelliğidir, $Breeze^t$ ise ajanın o zamandaki algısıdır.

Her t ve [x,y] için şu bağlantı aksiyomlarını ekleyebiliriz:

$$\boxed{L^t_{x,y} \Rightarrow (Breeze^t \Leftrightarrow B_{x,y})}$$

$$\boxed{L^t_{x,y} \Rightarrow (Stench^t \Leftrightarrow S_{x,y})}.$$

Bu ifadeler, "ajan burada durdu ve koku aldı" bilgisini "bu kare kokuludur" türünden kalıcı çevre bilgisiyle ilişkilendirir.

Eylem oluşumu ve etki aksiyomları (Effect axioms)

Ajanın eylemleri de zamanla indekslenir: $Forward^0$ zaman 0'da ileri gitmek demektir. Standart sıralama şöyledir: **t zamanında algı → t zamanında eylem → t+1 zamanındaki yeni durum**. Eylemin sonuçlarını ifade eden **effect axioms** oluştururuz.

Örneğin zaman 0'da [1,1]'de doğuya bakan ajan `Forward` yaparsa zaman 1'de [2,1]'dedir ve artık [1,1]'de değildir:

$$\boxed{L^0_{1,1} \wedge FacingEast^0 \wedge Forward^0 \Rightarrow (L^1_{2,1} \wedge \neg L^1_{1,1}).} \tag{7.1}$$

Bu biçimde her t, 16 kare, dört yön ve `Forward`, `Grab`, `Shoot`, `Climb`, `TurnLeft`, `TurnRight` için kurallar yazılabilir. Böylece ajan $Forward^0$ bilgisini ekleyince $ASK(KB, L^1_{2,1}) = True$ elde edebilir.

Frame problem: Neler değişmedi?

Ancak $ASK(KB, HaveArrow^1)$ cevabı henüz kanıtlanamaz: ileri gitmek ajanın okunu ortadan kaldırmıyor olsa da **bunu söyleyen hiçbir aksiyom yoktur**. Ajan ne okun kaldığını ne de kalmadığını ispatlayabilir. "Bir eylem gerçekleşince değişmeyen her şeyi nasıl temsil edeceğiz?" sorusu **frame problem**dir.

Basit çözüm, her eylemin değiştirmediği her özelliği açıkça yazan **frame axioms** kullanmaktır:

$$Forward^t \Rightarrow (HaveArrow^t \Leftrightarrow HaveArrow^{t+1})$$

$$Forward^t \Rightarrow (WumpusAlive^t \Leftrightarrow WumpusAlive^{t+1}).$$

Ancak m eylem ve n fluent için bu aksiyomların büyüklüğü $O(mn)$ olur. Bu, **representational frame problem**in özel biçimidir. Gerçek dünyada değişken sayısı çok büyüktür; neyse ki bir eylem genellikle yalnızca küçük k sayıda özelliği etkiler (**locality/yerellik**). Amaç gösterimi

$O(mk)$, t adım ileri çıkarım maliyetini ise $O(kt)$ mertebesine yaklaştırmaktır (ikincisine **inferential frame problem** denir). Kitap frame adının fizikteki referans çerçevesi ve film karelerinde sabit kalan arka planla benzerliğine dikkat çeker.

Successor-state axiom: Ardıl durum aksiyomu

Eylem yerine **fluent'e** odaklanılır. F^{t+1} iki durumda doğru olabilir: (a) t'deki eylem F'yi doğru yapmıştır; (b) F t'de zaten doğrudur ve eylem onu yanlış yapmamıştır.

$$\boxed{F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t).}$$

Ok örneği: Yeniden ok yükleme eylemi olmadığı için:

$$\boxed{HaveArrow^{t+1} \Leftrightarrow (HaveArrow^t \wedge \neg Shoot^t).} \tag{7.2}$$

Başta ok varsa ve ateş edilmiyorsa sonraki adımda da vardır; ateş edilirse artık yoktur. `Forward` için ayrıca ok değişmiyor aksiyomu yazmaya gerek kalmaz.

Konum örneği: [1,1]'e sonraki adımda gelmek için [1,2]'den güneye veya [2,1]'den batıya gidilmiş olabilir; ya da ajan [1,1]'dedir ve ilerleme olmamıştır:

$$\begin{aligned} L_{1,1}^{t+1} \Leftrightarrow\ & (L_{1,1}^t \wedge (\neg Forward^t \vee Bump^{t+1})) \\ & \vee (L_{1,2}^t \wedge South^t \wedge Forward^t) \\ & \vee (L_{2,1}^t \wedge West^t \wedge Forward^t). \end{aligned} \tag{7.3}$$

Üç VEYA kolunun her biri [1,1]'de bulunmaya yol açan farklı durumdur. Diğer fluent'lerin aksiyomlarını yazmak kitabın 7.26 alıştırmasında istenir.

Tam algı/eylem geçmişinden güncel KB çıkarımı

Bütün ardıl durum ve ortam aksiyomları eklenince ajan mevcut dünya hakkındaki cevaplanabilir soruları sorabilir. İlk örnekteki algı/eylem dizisi (sıra: koku, esinti, parıltı, çarpma, çığlık):

Zaman	Algı	Bu algıdan sonraki eylem
0	`[None,None,None,None,None]`	`Forward`^{0}
1	`[None,Breeze,None,None,None]`	`TurnRight`^{1}
2	`[None,Breeze,None,None,None]`	`TurnRight`^{2}
3	`[None,Breeze,None,None,None]`	`Forward`^{3}
4	`[None,None,None,None,None]`	`TurnRight`^{4}
5	`[None,None,None,None,None]`	`Forward`^{5}
6	`[Stench,None,None,None,None]`	Sorgu/sonraki karar

Örneğin sıfırıncı algı: $\neg Stench^0 \wedge \neg Breeze^0 \wedge \neg Glitter^0 \wedge \neg Bump^0 \wedge \neg Scream^0$. Her adımda bütün algı bileşenleri kaydedilir. Sonuçta:

$$ASK(KB, L_{1,2}^6) = True, \quad ASK(KB, W_{1,3}) = True, \quad ASK(KB, P_{3,1}) = True.$$

Ajan konumunu, Wumpus'u ve bir çukuru bulmuştur. En önemli hareket güvenliği kuralı:

$$\boxed{OK_{x,y}^t \Leftrightarrow \neg P_{x,y} \wedge \neg (W_{x,y} \wedge WumpusAlive^t).}$$

Kare çukur içermemeli, ayrıca içinde **canlı** Wumpus olmamalıdır. Ölü Wumpus'lu kare, çukur yoksa geçilebilir. Bu örnekte $ASK(KB, OK^6_{2,2}) = True$ bulunur. Sağlam ve tam DPLL benzeri yöntemler küçük-orta ölçekli Wumpus dünyalarında bu soruları çok kısa sürede yanıtlayabilir.

Qualification problem: Eylemin istisnalarını ne kadar ayrıntılandıracağız?

Forward duvar yoksa hareket eder dedik, fakat ajan tökezleyebilir, hastalanabilir veya dev yarasalar tarafından sürüklenebilir! Eylemin istenen sonucu vermesi için **bütün önkoşulları ve istisnaları** sayma sorunu **qualification problem**dir. Mantık içinde eksiksiz genel çözümü yoktur; model tasarımcısı ne kadar ayrıntı temsil edeceğini seçmelidir. 13. bölümde olasılık kuramının çok sayıda istisnayı tek tek adlandırmadan özetleyebilmesi ele alınacaktır.

7.7.2 — Hibrit ajan (A hybrid agent)

Dünyanın çeşitli özelliklerini çıkarmak için mantık, kararları uygulamak için **koşul–eylem kuralları** ve 3.–4. bölümlerin **arama/planlama algoritmaları** birleştirilirse **hibrit ajan** oluşur. Şekil 7.20'deki ajan hem bilgi tabanı hem de bir plan (eylem dizisi) tutar. Başlangıç KB'si zamandan bağımsız "Wumpus fiziği" kurallarını içerir; her zaman adımında yeni algılarla zamanlı successor-state aksiyomları eklenir. ASK ile güvenli ve ziyaret edilmeyen kareler belirlenir.

Şekil 7.20 — Hibrit Wumpus ajanı

```
function HYBRID-WUMPUS-AGENT(percept) -> action
  kalıcı: KB   = zamansız Wumpus aksiyomları
          t    = 0
          plan = boş eylem listesi

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  t zamanına ait zamansal aksiyomları TELL ile ekle
  safe <- {[x,y] : ASK(KB, OK^t_x,y) = true}

  if ASK(KB, Glitter^t) = true:
    plan <- [Grab] + PLAN-ROUTE(current, {[1,1]}, safe)
                   + [Climb]

  if plan boş:
    unvisited <- henüz ziyaret edildiği bilinmeyen kareler
    plan <- PLAN-ROUTE(current, unvisited ∩ safe, safe)

  if plan boş and ASK(KB, HaveArrow^t) = true:
    possible_wumpus <- {[x,y] : ASK(KB, ¬W_x,y) = false}
    plan <- PLAN-SHOT(current, possible_wumpus, safe)

  if plan boş:
    not_unsafe <- {[x,y] : ASK(KB, ¬OK^t_x,y) = false}
    plan <- PLAN-ROUTE(current, unvisited ∩ not_unsafe, safe)

  if plan boş:
    plan <- PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
```

```
  action <- POP(plan)
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t <- t + 1
  return action

function PLAN-ROUTE(current, goals, allowed) -> action list
  problem <- ROUTE-PROBLEM(current, goals, allowed)
  return A*-GRAPH-SEARCH(problem)
```

Ajanın **hedef öncelikleri** şöyledir:

1. Altın parlıyorsa **Grab**, güvenli yoldan [1,1]'e dönüş, **Climb**.
2. Mevcut plan yoksa ziyaret edilmemiş en yakın **kesin güvenli** kareye güvenli rota.
3. Güvenli keşif kalmadıysa ve oku varsa Wumpus olabilecek konumlardan birine atış planı (`PLAN-SHOT`, uygun atış hattına rota kurar).
4. Hiçbir güvenli seçenek yoksa, **tehlikeli olduğu kanıtlanamayan** karelere doğru sınırlı riskli keşif.
5. O da mümkün değilse [1,1]'e dönerek çıkış.

Rota planlamasında **A*** kullanılır; bu hesaplamayı ASK yapmaz. Mantık güvenli kareleri **kanıtlar**, arama algoritması bunların içinde yürünebilir yol bulur. `ASK(KB, ¬W_x,y)=false` ifadesi "Wumpus olmadığı ispatlanamıyor" demektir; orada Wumpus'un kesin olduğu anlamına gelmez. Benzer şekilde `ASK(KB, ¬OK)=false` **kesin güvenli** değil, yalnızca **kesin tehlikeli olduğu bilinmiyor** demektir.

7.7.3 — Mantıksal Durum Tahmini (Logical State Estimation)

Hibrit ajanın önemli zayıflığı şudur: zaman geçtikçe ASK sorgularının maliyeti artar; eski algı ve eylemlere dönerek giderek daha fazla sembol üzerinden çıkarım yapmak gerekir. Her algı için işlem süresinin ajanın ömrüyle birlikte büyümesi sürdürülemez. İstenen, t'den mümkün olduğunca bağımsız **sabit güncelleme zamanı**dır. Bunun için önceki çıkarımlar saklanabilir (**cache/önbellek**).

4.4. bölümdeki gibi geçmiş algıların bütün etkilerini, mevcut zamanda **mümkün olan durumların kümesi** olan **belief state (inanç durumu)** ile temsil edebiliriz. Yeni algılar geldikçe bu kümenin güncellenmesine **state estimation (durum tahmini)** denir. 4.4'te bu durumlar açıkça listeleniyordu; şimdi zamanlı ve zamansız sembolleri içeren **mantıksal cümle** kullanabiliriz:

$$\boxed{WumpusAlive^1 \wedge L^1_{2,1} \wedge B_{2,1} \wedge (P_{3,1} \vee P_{2,2}).} \tag{7.4}$$

Bu cümle zaman 1'de Wumpus'un hayatta olduğu, ajanın [2,1]'de bulunduğu, esinti olduğu ve [3,1] ya da [2,2]'de **en az bir çukur** bulunduğu bütün dünyaları temsil eder.

Exact belief state neden pahalıdır?

Zaman t'de n fluent varsa fiziksel durum sayısı 2^n olur. Belief state, fiziksel durumların **herhangi bir alt kümesi** olabilir; dolayısıyla farklı belief state sayısı:

$$\boxed{2^{2^n}}$$

olur. Bir belief state'i benzersiz ikili etiketle belirtmek için en az $\log_2(2^{2^n}) = 2^n$ bit gerekebilir. Yani en verimli kodlamada bile bazı tam belief state gösterimleri n ile **üstel büyür**. Örneğin n=3 iken 8 fiziksel durumun $2^8 = 256$ farklı alt kümesi vardır; n=5 iken 32 durumun 2^{32} alt kümesi vardır.

Yaklaşık belief state olarak 1-CNF

Yaygın bir yöntem, belief state'i **literallerin VE'si** olarak tutmaktır; buna **1-CNF** denir. Ajan önceki belief state'den her güncel X^t için X^t ve $\neg X^t$ sorgularını ayrı ayrı ispatlamaya çalışır; ayrıca henüz bilinmeyen zamansız sembolleri sorgular. İspatlanabilen literaller yeni belief state'te birleştirilir, eski belief state atılır.

Bu yöntem bazen bilgi kaybeder. Denklem 7.4'te $P_{3,1} \vee P_{2,2}$ bilinir, ama tek başına $P_{3,1}$ veya $P_{2,2}$ ispatlanamaz. Böylece **"en az biri çukur"** bilgisi 1-CNF'de saklanmayabilir. Kitap 7.27 alıştırmasında bu tür kayıpların olası çözümlerini araştırır.

Şekil 7.21 — Korumacı yaklaşım

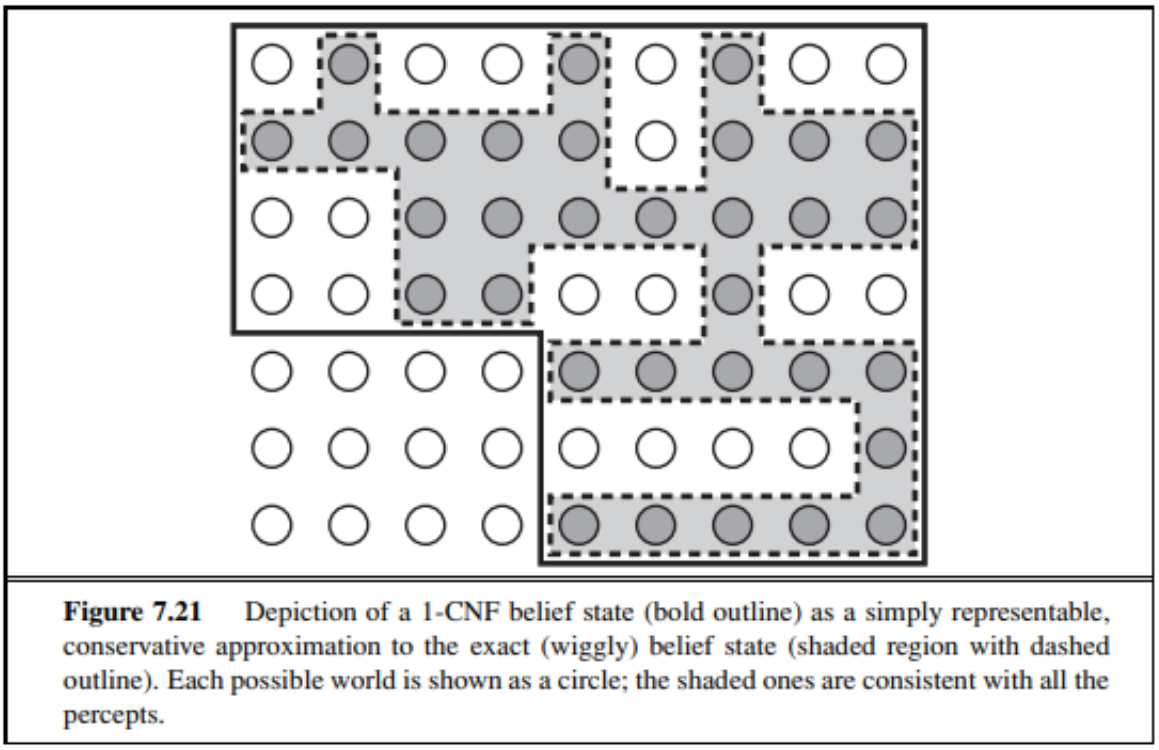


Figure 7.21 Depiction of a 1-CNF belief state (bold outline) as a simply representable, conservative approximation to the exact (wiggly) belief state (shaded region with dashed outline). Each possible world is shown as a circle; the shaded ones are consistent with all the percepts.

Şekil 7.21 — Dairelerin her biri olası dünyadır. Kesikli kıvrımlı bölgedeki gri daireler algılarla gerçekten uyumlu durumlardır. Kalın düz sınır, 1-CNF'nin temsil ettiği daha geniş olasılıklar kümesidir.

Başlangıç belief state doğruysa ve her yeni literal sound çıkarımla ispatlanıyorsa, 1-CNF temsilindeki bütün literaller gerçektir. Dolayısıyla gerçek olası durumların **hiçbiri kaybedilmez**, ancak gereksiz ek durumlar mümkün sayılabilir. Matematiksel olarak:

$$B_{exact} \subseteq B_{approx}.$$

Buna **conservative approximation (korumacı yaklaşım)** denir: kesin mümkün olanların tamamını kapsayan "dış zarf". Karmaşık kümeleri güvenle yaklaşık temsil etmek yapay zekânın birçok alanında tekrar karşımıza çıkar.

Öğretici örnek: Gerçekte üç olası tehlike düzeni kalmışken yaklaşık temsil beş düzene izin verebilir. Bilgi daha zayıftır ama gerçek üç seçenek yanlışlıkla dışlanmamıştır.

7.7.4 — Önermesel Çıkarımla Plan Yapma

Hibrit ajan güvenliği mantıkla saptıyor, planı A* ile kuruyordu. Şimdi **planı da mantıksal doyurulabilirlik** problemi olarak ifade edeceğiz:

1. $Init^0$ başlangıç bilgileri, her zaman ve eylem için $Transition^1, ..., Transition^t$ ardıl durum kuralları ve t zamanındaki hedef $HaveGold^t \wedge ClimbedOut^t$ içeren bir mantıksal cümle kurulur.
2. Bu cümle **SAT solver**a verilir. Doyurucu model bulunursa belirtilen t adımında hedefe ulaşılabilir; cümle doyurulamazsa o t için plan yoktur.
3. Bulunan modelde doğru atanmış **eylem değişkenleri** zaman sırasına konarak plan çıkarılır.

Şekil 7.22 — SATPLAN

```
function SATPLAN(init, transition, goal, Tmax)
  for t = 0 to Tmax:
    cnf <- TRANSLATE-TO-SAT(init, transition, goal, t)
    model <- SAT-SOLVER(cnf)
    if model != null:
      return EXTRACT-SOLUTION(model)
  return failure
```

SATPLAN planın kaç adım süreceğini baştan bilmediği için `t=0,1,2,...,Tmax` sırayla dener. Bu sayede **verilen sınır içinde var olan en kısa eylem dizisi** bulunur (birim eylem süreleri ve doğru modelleme varsayımlarıyla). Ancak bu biçimi kısmen gözlemlenebilir dünyaya doğrudan uygulanamaz: bilinmeyen değişkenlere SAT çözücü, çözüm bulmasını kolaylaştıran değerler atayabilir; bu gerçek dünyada garantili başarı demek değildir.

Neden SATPLAN bazen “Shoot” ile yer değiştirmiş gibi görünüyor?

Başlangıç bilgisi $L^0_{1,1}$ ve hedef $L^1_{2,1}$ olsun. `Forward^0` ile hedefe ulaşılması beklenir. Fakat yetersiz aksiyomlarla SATPLAN `Shoot^0` seçebilir; çünkü model içinde $L^0_{2,1}$ değerini de doğru yapmıştır: ajanın **aynı anda iki yerde bulunması** açıkça yasaklanmamıştır!

ASK ile SAT arasındaki fark: ASK için $L^0_{2,1}$ ispatlanamamışsa onu ispatın dayanağı olarak kullanamayız. SAT içinse bu sembol **atanmamıştır** ve hedefi sağlayacak biçimde doğru yapılabilir. Bu nedenle SATPLAN, bilgi tabanındaki eksik kısıtları ortaya çıkaran bir **hata ayıklama aracı** olarak da yararlıdır.

Konum kısıtı: Her t’de tam olarak bir konum doğru olmalı. Bunu “en az bir” konum ve “farklı iki konum birlikte olamaz” clause’larıyla ifade edebiliriz; ya da başlangıçta [1,1] dışındaki bütün $\neg L^0_{x,y}$ aksiyomlarını ekleyip sonraki zamanları successor-state kurallarıyla yönetebiliriz. Aynısı **tek yönelim** için de geçerlidir.

Önkoşul aksiyomları (Precondition axioms)

SATPLAN, ok yokken ateş etmek gibi imkânsız eylemleri de seçebilir. Successor-state kuralı başarısız eylemin bir etkisinin olmadığını belirtse bile **eylemin seçilmesini yasaklamayabilir**. Bu nedenle eylem için gerekli koşulu açıkça yazmalıyız:

$$\boxed{Shoot^t \Rightarrow HaveArrow^t.}$$

Ajanın oku yoksa Shoot seçimi formülü yanlış yapar. Önkoşul aksiyomları böylece geçersiz planları önler. Önkoşullar ayrıca successor-state kurallarında yeniden uzun uzun yazılmak zorunda kalmaz.

Eylem dışlama aksiyomları (Action-exclusion axioms)

SATPLAN bazı modellerde $Forward^0$ ve $Shoot^0$ eylemlerini aynı anda doğru yapabilir. Tek eylemli Wumpus ajanında buna izin verilmez. Her eşzamanlı yapılamayan eylem çifti için:

$$\boxed{\neg A_i^t \vee \neg A_j^t}$$

aksiyomu eklenir. Gerçek dünyada ileri yürürken ateş etmek mümkün, ateş ederken nesne tutmak başka bağlamda mümkün olmayabilir. Yalnız **gerçekten birbiriyle çatışan** eylemleri dışlarsak, çok eylemli eşzamanlı planlamaya da izin verebiliriz; SATPLAN süre açısından daha kısa planları kullanabilir.

SATPLAN'in doğru tanımı ve 11 adımlık örnek

Geçerli planlama formülünde **başlangıç durumu, hedef, successor-state aksiyomları, eylem önkoşulları ve eylem dışlama kuralları** bir arada bulunur; ayrıca ajan konum/yön gibi tek-değer kısıtları eksiksiz tanımlanır. Kitap, bu tür aksiyomlarla üretilen doyurucu modellerin gerçek problem için geçerli planlara karşılık geldiğini belirtir. DPLL türü SAT çözücü, Şekil 7.2'deki Wumpus örneği için **11 adımlık plan** elde edebilir.

Önermeler mantığı neden bir noktada yetmez?

"Her t için" ve "her [x,y] için" biçimindeki şablonlar, önermeler mantığında genel tek cümle olarak ifade edilemez; program bunlardan her kare ve zaman için **ayrı aksiyom üretmek** zorundadır. 100×100 kare ve 1000 zaman adımına sahip bir dünya, onlarca veya yüzlerce milyon cümle gerektirebilir. Oysa dünyanın fizik kuralları her karede ve zamanda aynıdır. Bu genel ilişkiyi doğrudan ifade eden daha güçlü bir dile gerek vardır: **Birinci dereceden mantık (first-order logic)**. Bir sonraki ünite, farklı dünya boyutlarını temsil etmek için milyonlar yerine az sayıda genel cümle yazabilmemizi sağlayan dili tanıtır.

7. Ünitenin Tek Paragraflık Sonucu

Mantıksal ajan, çevresinden algıları alır, zamana ve dünyaya ilişkin kuralları bilgi tabanında tutar ve bu bilgilerin zorunlu sonuçlarını çıkarır. $\models$ ilişkinin anlamsal doğruluğunu, $\vdash$ kullanılan ispat mekanizmasını gösterir. Önermeler mantığında doğruluk tabloları, Resolution, Horn zincirlemesi, DPLL ve WalkSAT farklı görevleri yerine getirir. Zamanla değişen bilgileri successor-state aksiyomlarıyla koruyabilir, hibrit ajanlarda mantıksal güvenliği arama algoritmasıyla birleştirebilir ve SATPLAN ile plan arayabiliriz. Büyük ve genel dünyalarda dilin ifade gücü sorun hâline geldiğinde birinci dereceden mantığa geçeriz.

EK A — Temel Kavramlar ve Sembol Sözlüğü

İngilizce	Türkçe karşılık	Bu ünitedeki işlev
Knowledge-based agent	Bilgi tabanlı ajan	Kurallar/olgularla hareket seçer
Knowledge base (KB)	Bilgi tabanı	Mantıksal cümleleri tutar
Axiom	Aksiyom	Başlangıçta kabul edilen bilgi
Sentence / literal	Cümle / literal	Biçimsel önerme / atomik önerme veya olumsuzu
TELL / ASK	Bilgi ekleme / sorgu	Ajanın KB arayüzü
Inference / reasoning	Çıkarım / akıl yürütme	Verilen bilgilerden yeni bilgiye geçiş
Syntax / semantics	Sözdizimi / anlambilim	İfade biçimi / doğruluk anlamı
Model / satisfaction	Model / doyurma	Değer ataması / cümleyi doğru kılma
Entailment	Mantıksal gerektirme	Her KB modelinde hedefin doğruluğu
Soundness / completeness	Sağlamlık / tamlık	Geçersiz sonuç üretmeme / bütün geçerli sonuçlara erişebilme
Grounding	Temellendirme	Temsilin gerçek dünyaya bağlanması
Tautology	Totoloji	Bütün modellerde doğru cümle
SAT / unsatisfiable	Doyurulabilirlik / doyurulamazlık	Uygun model var mı / hiçbir model yok mu?
Proof by refutation	Çürütmeyle ispat	KB ve hedefin olumsuzundan çelişki bulma
Modus Ponens	Koşullu çıkarım	P⇒Q ve P'den Q
Resolution / resolvent	Çözümleme / çıkan clause	Tamamlayıcı literalleri eleme
CNF	Bağlaçsal normal biçim	Clause'ların VE birleşimi
Horn / definite clause	Horn / kesin clause	En fazla bir / tam bir pozitif literal
Forward chaining	İleri zincirleme	Olgulardan sonuç üretme
Backward chaining	Geri zincirleme	Hedeften olgulara geriye ispat
DPLL / WalkSAT	Tam SAT / yerel SAT araması	Doyurulabilirlik kararı / uygun model arama
Fluent / atemporal	Zamanla değişen / değişmeyen	Konum/ok gibi durum / sabit mağara özellikleri
Successor-state axiom	Ardıl durum aksiyomu	Bir fluent'in t+1 değerini açıklar
Frame problem	Değişmeyenleri temsil sorunu	Eylem sonrası neyin aynı kaldığını izleme
Qualification problem	Eylem koşullarının eksiksizliği sorunu	Bütün istisnaları nasıl sayacağız?
Belief state	İnanç durumu	Bütün olası mevcut durumlar kümesi
Conservative approximation	Korumacı yaklaşım	Gerçek olası durumları kapsam dışı bırakmama
SATPLAN	SAT tabanlı planlayıcı	Eylem dizisi için doyurucu model arama

Formül Rehberi

Amaç	Kısa gösterim
Mantıksal gerektirme	$KB \models \alpha$
Model kümesi ile gerektirme	$M(KB) \subseteq M(\alpha)$
i yöntemiyle çıkarım	$KB \vdash_i \alpha$
Koşulun eşdeğeri	$P \Rightarrow Q \equiv \neg P \vee Q$
Çift koşulun eşdeğeri	$P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$
De Morgan 1	$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
De Morgan 2	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
Çelişkiyle gerektirme	$KB \models \alpha \iff KB \wedge \neg\alpha$ doyurulamaz
Olası model sayısı	2^n
Wumpus güvenliği	$OK^t_{x,y} \Leftrightarrow \neg P_{x,y} \wedge \neg(W_{x,y} \wedge WumpusAlive^t)$
Okun devamı	$HaveArrow^{t+1} \Leftrightarrow HaveArrow^t \wedge \neg Shoot^t$

EK B — Adım Adım Çözümlü Alıştırmalar

Buradaki alıştırmalar kitap metnindeki alıştırmaların yerine geçmesi için değil, ünitenin temel kavramlarını çalıştırmak üzere **ek öğretici örnekler** olarak hazırlanmıştır.

Alıştırma B.1 — Sözdizimi mi, doğruluk mu?

`(P ∧ Q)` ve `P ∧ ⇒ Q` ifadelerini değerlendir. Ayrıca "4+3=10" için sonuç nedir?

Çözüm: `(P ∧ Q)` iyi biçimli mantıksal cümledir; doğru/yanlış olduğu modele bağlıdır. `P ∧ ⇒ Q` iyi biçimli değildir. `4+3=10` aritmetik sözdizimine uygundur ama yanlıştır. **Ders:** iyi biçimlilik ile doğruluk karıştırılmaz.

Alıştırma B.2 — Model sayısı

Beş bağımsız önerme sembolünün kaç farklı doğru/yanlış ataması vardır? On değişken olursa?

Çözüm: $2^5 = 32$ ve $2^{10} = 1024$ atama vardır. Her yeni bağımsız sembol olasılık sayısını ikiye katlar.

Alıştırma B.3 — Koşullu önermenin özel durumu

P yanlış, Q yanlışken $P \Rightarrow Q$ doğru mudur?

Çözüm: Evet. Koşullu önerme yalnızca P doğru ve Q yanlışken yanlıştır. Eşdeğer biçimi $\neg P \vee Q$: $True \vee False = True$.

Alıştırma B.4 — XOR ile OR

P ve Q'nun ikisi de doğruysa $P \vee Q$ ve $P \oplus Q$ sonuçları nedir?

Çözüm: Normal VEYA doğru; özel VEYA (XOR) yanlış. Normal VEYA "en az biri", XOR "tam birisi" anlamına gelir.

Alıştırma B.5 — De Morgan

$\neg(A \vee (B \wedge C))$ ifadesini olumsuzlamaları atomlara taşıyacak biçimde dönüştür.

Çözüm: Önce $\neg A \wedge \neg(B \wedge C)$; sonra $\boxed{\neg A \wedge (\neg B \vee \neg C)}$.

Alıştırma B.6 — Gerekli ama yeterli değil

KB, $P \vee Q$ ve $\neg P$ içeriyor. Q sonucu kesin midir?

Çözüm: Evet. P yanlışsa P veya Q ifadesinin doğru kalabilmesi için Q doğru olmalıdır. Resolution ile $(P \vee Q), \neg P \vdash Q$.

Alıştırma B.7 — Wumpus'un ilk iki algısı

[1,1]'de esinti yok; [2,1]'de esinti var. [2,2]'de kesin çukur olduğunu söyleyebilir miyiz?

Çözüm: Hayır. $P_{2,2} \vee P_{3,1}$ doğrudur ama iki literalden hangisinin doğru olduğu henüz bilinmez. [2,2]'de çukur bulunmadığı da kesin değildir.

Alıştırma B.8 — Wumpus'ta yeni algı

[1,2]'de esinti algılanmıyor. Önceden $P_{2,2} \vee P_{3,1}$ biliniyordu. Sonucu yaz.

Çözüm: [1,2]'de esinti yoksa [2,2]'de çukur yoktur ($\neg P_{2,2}$). Bu bilgiyle VEYA'dan geriye $\boxed{P_{3,1}}$ kalır.

Alıştırma B.9 — Soundness mı completeness mı?

Bir algoritma hiçbir zaman yanlış çıkarım yapmıyor ama bazı geçerli sonuçları bulamıyor. Hangi özelliklere sahip?

Çözüm: **Sound**dur; **complete** değildir. Yanlış sonuç üretmemek ile her doğru sonucu bulmak farklı özelliklerdir.

Alıştırma B.10 — SAT mi, validity mi?

$P \wedge Q$ ifadesi doyurulabilir mi? Geçerli mi? $P \wedge \neg P$ için aynı iki soruyu cevapla.

Çözüm: $P \wedge Q$ için P=Q=True bir modeldir: **doyurulabilir**, ama her modelde doğru olmadığından **geçerli değil**. $P \wedge \neg P$ hiçbir modelde doğru değildir: **doyurulamaz ve geçerli değil**.

Alıştırma B.11 — Modus Ponens ve And-Elimination

$(A \wedge B) \Rightarrow C$ ve $A \wedge B$ veriliyor. Önce hangi sonuç, sonra hangi alt bilgiler elde edilir?

Çözüm: Modus Ponens ile C. And-Elimination ile A ve B ayrı ayrı da elde edilebilir.

Alıştırma B.12 — CNF dönüşümü

$A \Rightarrow (B \wedge C)$ ifadesini CNF'ye çevir.

Çözüm: $\neg A \vee (B \wedge C)$ → dağılımla $\boxed{(\neg A \vee B) \wedge (\neg A \vee C)}$.

Alıştırma B.13 — Horn mu, değil mi?

Şu clause'ları sınıflandır: (i) $\neg A \vee \neg B \vee C$ (ii) $A \vee B$ (iii) $\neg A \vee \neg B$.

Çözüm: (i) tam bir pozitif içerir: definite ve Horn. (ii) iki pozitif içerir: Horn değil. (iii) pozitif literal yoktur: goal clause ve Horn, ama definite değil.

Alıştırma B.14 — Forward chaining

Facts A,B; kurallar $A \wedge B \Rightarrow C$, $C \Rightarrow D$, $D \Rightarrow E$. E nasıl ispatlanır?

Çözüm: A ve B biliniyor → C; C → D; D → E. Algoritma olgulardan hedefe doğru ilerler.

Alıştırma B.15 — Backward chaining

B.14'te sadece "E doğru mu?" sorusu sorulursa geriye doğru hangi alt hedefler oluşur?

Çözüm: E için D, D için C, C için A ile B gerekir. A ve B fact olduğu için bütün alt hedefler ispatlanır ve E doğrudur.

Alıştırma B.16 — Saf sembol bul

CNF clause'ları `(A ∨ ¬B)`, `(A ∨ C)`, `(¬B ∨ ¬C)`. Hangi semboller saf?

Çözüm: A yalnız pozitif, B yalnız negatif görünür: saf. C iki işaretle görünür: saf değil.

Alıştırma B.17 — Unit propagation

$B = True$ biliniyor, `(¬B ∨ ¬C)` ve `(C ∨ A)` var. Zorunlu atamaları sırayla yaz.

Çözüm: `(¬B ∨ ¬C)` → ¬C, dolayısıyla C=False. `(C ∨ A)` → A, dolayısıyla A=True. Dallanma gerekmez.

Alıştırma B.18 — WalkSAT sonucu nasıl yorumlanır?

WalkSAT 100.000 flip sonunda model bulamadı. Formül doyurulamaz mı?

Çözüm: Kanıtlanamaz. Sınırlı yerel arama başarısızlığı, çözüm olmadığı veya yeterince şans/zaman olmadığı anlamına gelebilir. Tam bir SAT karar yöntemi gereklidir.

Alıştırma B.19 — Zamana bağlı değişken

Ajan t=0'da [1,1], t=1'de [2,1] konumunda. $L^0_{1,1}$ ve $L^1_{2,1}$ çelişkili midir?

Çözüm: Hayır. Farklı zamanları anlatırlar. Zamansız L sembolleri kullanmak yanlış izlenim doğurabilirdi.

Alıştırma B.20 — SATPLAN neden yanlış eylem seçer?

Ajanın oku yok fakat SATPLAN `Shoot` seçiyor. Hangi aksiyom eksiktir?

Çözüm: Eylemin **önkoşulu**: $Shoot^t \Rightarrow HaveArrow^t$. Sadece eylemin etkisini tanımlamak, eylemi geçersiz kılmaya yetmeyebilir.

Alıştırma B.21 — Conservative approximation

Tam belief state üç dünya içeriyor; yaklaşık belief state bu üçünün yanında iki ek dünya içeriyor. Korumacı yaklaşım mı?

Çözüm: Evet, çünkü $B_{exact} \subseteq B_{approx}$. Ek dünyalar belirsizliği artırır ama gerçek olası dünya dışlanmaz.

Alıştırma B.22 — Neden birinci dereceden mantık?

"Her karede, komşuda çukur varsa o kare esintilidir" kuralını önermeler mantığında yazmak neden ölçeklenmez?

Çözüm: Önermeler mantığında “bütün x,y için” şeklinde niceleme olmadığından her kare/zaman için ayrı örnek cümle üretmek gerekir. Birinci dereceden mantık genel kuralı tek şablonla ifade etmeye izin verir.

Hızlı Tekrar İçin Sınav Öncesi Kontrol Listesi

- ☐ KB, TELL, ASK ve inference kavramlarını açıklayabiliyorum.
- ☐ Wumpus’un beş algısını ve altı temel eylemini biliyorum.
- ☐ Koku/esinti yokluğundan güvenli komşu çıkarabiliyorum.
- ☐ Syntax, semantics, model, entailment arasındaki farkı kurabiliyorum.
- ☐ $\models$ ile $\vdash$ arasındaki farkı biliyorum.
- ☐ Beş bağlacın doğruluk tablosunu hesaplayabiliyorum.
- ☐ R1–R5 bilgi tabanının neden yalnızca üç model sağladığını gösterebiliyorum.
- ☐ Soundness ile completeness’i karıştırmıyorum.
- ☐ Geçerlilik, doyurulabilirlik, doyurulamazlık örnekleri verebiliyorum.
- ☐ Modus Ponens, Resolution ve CNF dönüşümü yapabiliyorum.
- ☐ Definite ve Horn clause’u pozitif literal sayısıyla ayırabiliyorum.
- ☐ Forward ve backward chaining’i küçük örnek üzerinde çalıştırabiliyorum.
- ☐ DPLL ile WalkSAT’in hangisinin doyurulamazlığı kanıtlayabildiğini biliyorum.
- ☐ Fluent, successor-state axiom ve frame problem’i açıklayabiliyorum.
- ☐ Hibrit ajan, belief state ve SATPLAN’in görevlerini ayırabiliyorum.

Kaynak ve çalışma kapsamı: Stuart Russell ve Peter Norvig, *Artificial Intelligence: A Modern Approach*, 3. baskı, Bölüm 7 “Logical Agents” (bu çalışma için sağlanan İngilizce metin ve şekiller). Türkçe anlatım, algoritma açıklamaları, sayısal çalışmalar, tablolar, yeni öğretici diyagramlar ve ek alıştırmalar bu ders notu için düzenlenmiştir. Özgün şekiller, gönderenin paylaştığı görüntülerden alınmıştır.